(The Fixers)

# (Flow Fund AI: Personal Finance Portfolio)

Software Engineering Project

Name (s): Ayan Lakhani, Vrushti Patel, Kirtan Patel, Quang Tran, Vishal Venu Bangrae

Semester: Spring 2026

Group Number: 9

Coordinator: Vishal Venu Bangrae

Name of the Guide: Dr. Tushara Sadasivuni

Date: (02/02/2026)

# TABLE OF CONTENTS


**1.0 INTRODUCTION** ..... 4

1.1 Software Engineers' information ..... 4

1.2 Planning and Scheduling ..... 4

1.3 Teamwork basics ..... 4

1.4 Problem Statement ..... 4

1.5 System Requirements ..... 5

1.5.1 Context Diagram ..... 7

1.5.2 Activity Diagram ..... 8

**2.0 REQUIREMENTS** ..... 10

2.1 Use Cases ..... 10

2.2 Requirements ..... 15

2.3 Use Case Diagrams ..... 20

**3.0 DATABASE TABLES** ..... 24

**4.0 CLASS DIAGRAM** .....

**5.0 BEHAVIORAL MODELING** .....

**6.0 IMPLEMENTATION** ..... 30

**7.0 TESTING** .....

7.1 Test cases .....

7.2 Testing .....

**8.0 ARCHITECTURAL MODELING** .....

8.1 Architectural Views .....

8.2 Architectural Model .....

**9.0 GITHUB** .....

REFERENCES .....

# 1.0 INTRODUCTION

## 1.1 Software Engineers' information

Ayan Lakhani: A full-stack software engineer with experience building end-to-end applications that combine backend logic, databases, and modern frontends. Strong interest in AI-assisted systems and automation, using Python and data-driven approaches to improve workflows and decision-making. Experienced in developing scalable solutions with cloud-ready architectures and real-world constraints in mind.

Vrushti Patel: I have a strong foundation in C/C++, Java, and Python. I am currently working to become a full-stack developer, focusing on building efficient and user-friendly website applications.

Kirtan Patel: I have strong skills in web programming and game development. I have experience building interactive web applications using modern frameworks and developing gameplay systems, while applying problem-solving skills to both software and game design.

Quang Tran: A full-stack SWE with hands-on experience in building and deploying production-ready, cloud-native applications. A strong interest in AI-integrated systems, utilizing Python, and AWS to automate. Experienced from React frontends to Node/Python backends and scalable databases.

Vishal Venu Bangare: A computer science student with strong skills in software engineering, data analysis, and backend development. Experienced in Python, Java, SQL, and full-stack web development, with hands-on work in building scalable applications and data pipelines. Strong interest in AI-assisted systems, analytics, and performance optimization, developed through internships, research, and hackathon projects.

## 1.2 Planning and Scheduling

| Assignee Name | Email address | Task | Duration (hours) | Dependency | Due Date | Evaluation |
|---|---|---|---|---|---|---|
| Ayan Lakhani | alakhani18@student.gsu.edu | Activity Diagram | 1 hr | Split with Vishal | 2/1/26 | 10/10 |
| Vrushti Patel | vpatel131@student.gsu.edu | Context Diagram | 1 hour | None | 2/1/26 | 9/10 |
| Kirtan Patel | kpatel315@student.gsu.edu | Problem Statements | 1 hour | Split with Quang | 2/1/26 | 9/10 |

| | | | | | | |
|---|---|---|---|---|---|---|
| Quang Tran (Coordinator) | qtran30@student.gsu.edu | Problem Statements | 2 hours | Split with Kirtan | 2/1/26 | 9/10 |
| Vishal Venu Bangrae | vbangrae1@student.gsu.edu | Activity Diagram | 1 hr | Split with Ayan | 2/1/26 | 10/10 |

## 1.3 Teamwork Basics

- We will be using Microsoft Teams for communication purposes and meetings.
- Everyone should check the group chat twice a day for any progress or questions.
- We should hold a meeting once a week on weekends if possible.
- Set deadlines and share everyone's progress in the meeting.
- We will divide the project into clear tasks and assign responsibilities based on each member's strengths.
- We will set internal deadlines ahead of the official due dates to allow time for review and revisions.
- We will use a rotating facilitator to keep the project on schedule and ensure meetings stay task-focused.
- We will review each deliverable as a group before submission and agree on quality standards.
- We will document decisions, changes, and progress so nothing gets lost between meetings.
- We will address missed deadlines or incomplete work immediately and redistribute tasks if necessary.
- We will use agreed-upon communication tools to share updates, files, and feedback efficiently.
- We will resolve disagreements by focusing on project requirements and reaching a consensus on the best solution

## 1.4 Problem Statement

FundFlow AI is a spending-aware investing platform designed to simplify investing for young adults and college students (ages 18-24) by reducing risk, confusion, and financial stress through education, safety, and intelligent automation. Unlike traditional investing or budgeting apps that treat spending and investing separately, FundFlow AI analyzes real spending behavior, income stability, budgeting habits, and risk tolerance to generate an AI-based investment readiness score before making any recommendation, ensuring users only invest when it is financially appropriate. Our platform does not autonomously trade; instead, it provides guided decision support by either recommending historically stable investment options or delivering personalized educational guidance to improve financial literacy and readiness. Targeted primarily at new investors and micro-investment users seeking smarter automation and secondarily at individuals who struggle with budgeting and financial awareness, the product addresses the common fear

that investing feels risky and overwhelming without a stable income or predictable expenses. Built with a React/React Native frontend, a Node.js/Express backend, Python-based ML services, and a secure MySQL data layer, the system integrates market data pipelines, an explainable AI readiness engine, and an AI chatbot with guardrails using RAG for transparent, compliant guidance. Strong security and fraud prevention measures, including MFA, role-based access, session security, encryption, audit logs, and anomaly detection, are core technical pillars. FundFlow AI differentiates itself from platforms like Acorns, Robinhood, and YNAB by combining spending awareness, financial readiness evaluation, and explainable investment guidance into one adaptive system, helping users build healthier investing habits early while maintaining confidence, transparency, and long-term financial stability.

**List of features:**

1. User Management
2. AI Financial Chatbot
3. Dashboard and analysis/ visualizations
4. Transaction Monitoring
5. AI investment readiness score engine
6. Investment simulations
7. Security/Privacy
8. Alerts and Notifications
9. Market Data Evaluation/Integration
10. Investment readiness module
11. Historical analysis
12. Goals/expectations and growth expectations

## 1.5 System Requirements

### 1.5.1 Context Diagram

**FlowFund AI** is a single, centralized system that interacts with other external systems and services required to calculate spending-aware investment analysis. This system is responsible for managing users, storing financial information, performing AI-driven analysis, delivering notifications or alerts, and getting financial data. The connected external systems are the User/Admin management module, Notifications/Alerts, Banking data system, Market data system, OpenAI/LLMs, and Financial data storage.

**User/Admin management module** will support user authentication, authorization, and role-based access control. It is made to identify users and admin roles, and enforce access restrictions. This will also safely store important users' information.

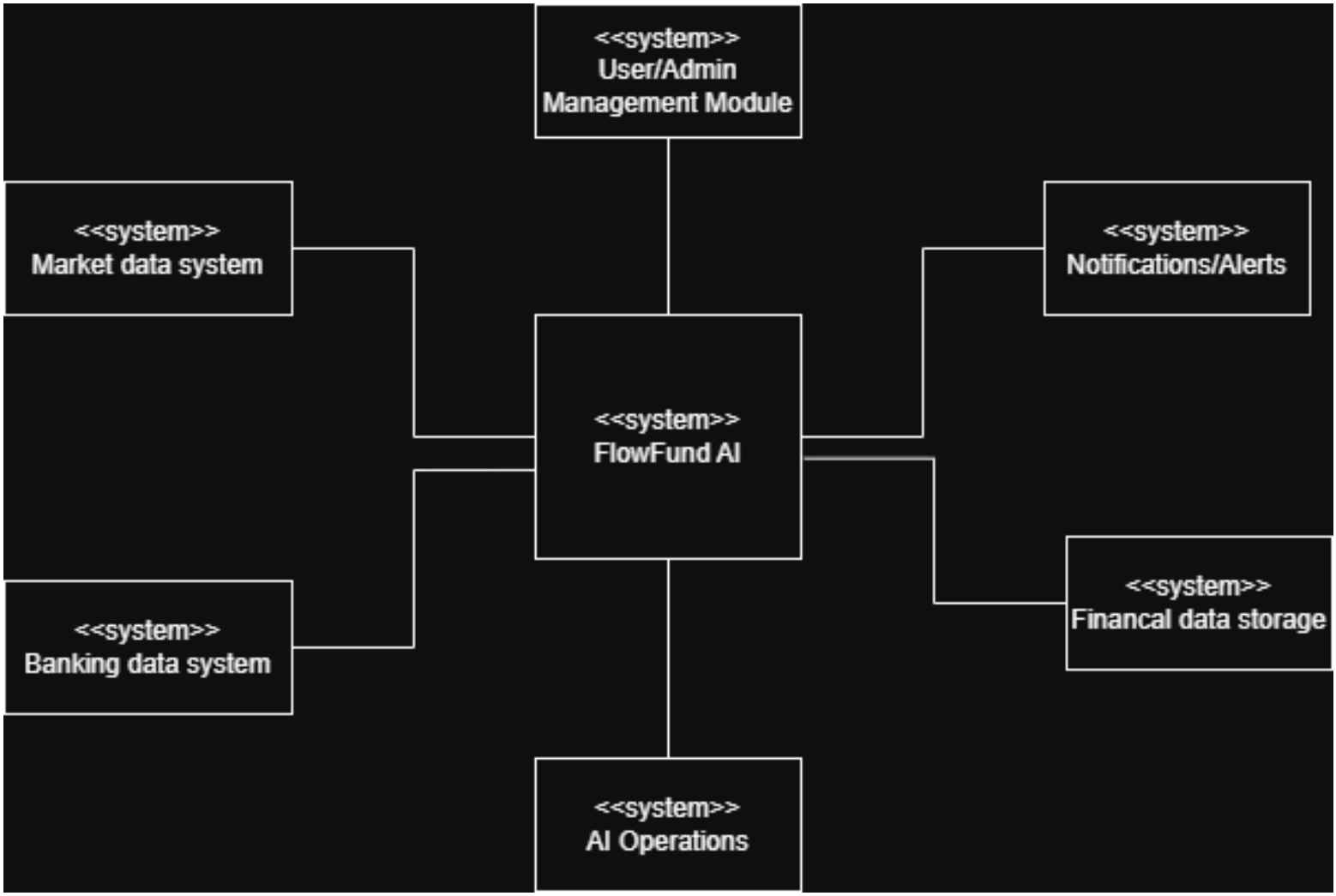
**Banking data system** will provide user transaction and account data through third-party financial aggregation services. Now, FlowFund AI will take this data to analyze spending behavior, calculate cash buffer, and support the investment readiness score engine.

**Market Data system** will give real-time market information, such as prices and investment trends. In addition to that, FlowFund AI will also use this data for investment simulations, growth projections, and offer explanations without performing real money transactions.

**OpenAI/ LLMs** will be used to send financial context and analysis results to these models to generate explanations, recommendations, and responses for the AI financial Chatbot and readiness report.

**Notifications and Alerts system** delivers system-generated messages to users regarding their readiness score changes, spending spikes, low cash warnings, and goal milestones.

**Financial Data System** will persist user profiles, transaction records, historical analysis, readiness score changes, and goal data. This storage will help with traceability and long-term analysis.



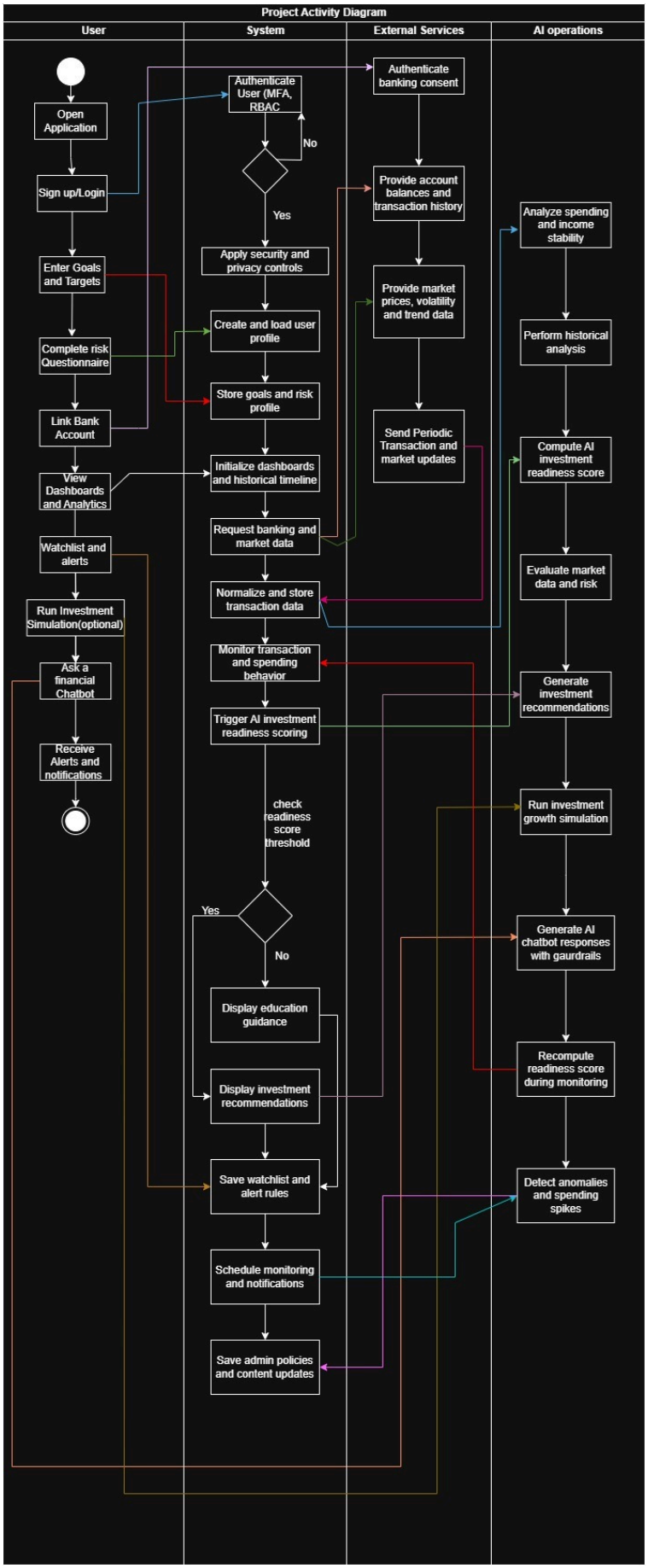
### 1.5.2 Activity Diagram

**User (Website User)**
The user interacts with the FlowFund web application to manage their personal investment journey. The user opens the application, signs up or logs in, sets financial goals and growth targets, completes a risk questionnaire, and links their bank account. Based on the insights provided by the system, the user can view dashboards and analytics, manage a personal watchlist, configure alerts, run optional investment simulations, and interact with the AI financial chatbot. The user also receives notifications and alerts related to spending behavior, readiness score changes, and market conditions.

**FlowFund System**
The FlowFund system serves as the core platform that coordinates all application functionality. It handles user authentication using MFA and role-based access control, enforces security and privacy controls, and manages user sessions. The system stores goals, risk profiles, and transaction data, initializes dashboards and historical timelines, and monitors transactions and spending behavior. It triggers AI investment readiness scoring, evaluates readiness thresholds, displays either educational guidance or investment recommendations, saves watchlists and alert rules, and schedules continuous monitoring and notifications. The system also persists administrative policy and content updates to ensure consistent platform behavior.

**External Services (Banking and Market Data Providers)**
External services supply critical financial data required for analysis and decision-making. Banking services authenticate user consent and provide account balances and transaction history, while market data services deliver asset prices, volatility metrics, and trend information. These services also send periodic transaction and market updates that enable continuous monitoring, alert evaluation, and up-to-date financial insights within the FlowFund platform.

**AI Operations**
AI operations are responsible for advanced analysis and intelligence within the system. These services analyze spending behavior and income stability, perform historical financial analysis, and compute the AI-based investment readiness score. They evaluate market data and risk factors, generate personalized investment recommendations, and run investment growth simulations based on user goals and risk tolerance. The AI services also generate chatbot responses with built-in guardrails, recompute readiness scores during ongoing monitoring, and detect anomalies or spending spikes that may trigger alerts or system actions.


Project Activity Diagram
User
System
External Services
AI operations
Open Application
Sign up/Login
Enter Goals and Targets
Complete risk Questionnaire
Link Bank Account
View Dashboards and Analytics
Watchlist and alerts
Run Investment Simulation(optional)
Ask a financial Chatbot
Receive Alerts and notifications
Authenticate User (MFA, RBAC
No
Yes
Apply security and privacy controls
Create and load user profile
Store goals and risk profile
Initialize dashboards and historical timeline
Request banking and market data
Normalize and store transaction data
Monitor transaction and spending behavior
Trigger AI investment readiness scoring
check readiness score threshold
Yes
No
Display education guidance
Display investment recommendations
Save watchlist and alert rules
Schedule monitoring and notifications
Save admin policies and content updates
Authenticate banking consent
Provide account balances and transaction history
Provide market prices, volatility and trend data
Send Periodic Transaction and market updates
Analyze spending and income stability
Perform historical analysis
Compute AI investment readiness score
Evaluate market data and risk
Generate investment recommendations
Run investment growth simulation
Generate AI chatbot responses with gaurdrails
Recompute readiness score during monitoring
Detect anomalies and spending spikes

| Assignee Name | Email address | Task | Duration (hours) | Dependency | Due Date | Evaluation |
|---|---|---|---|---|---|---|
| Ayan Lakhani | alakhani18@student.gsu.edu | Activity Diagram<br>Context Diagram | 2 hr | some group input | 2/23/26 | 10/10 |
| Vrushti Patel<br>(Coordinator) | vpatel131@student.gsu.edu | Database Management/ User Management Implementation | 3 hr | Worked with Quang and Kirtan. | 2/23/26 | 8/10 |
| Kirtan Patel | kpatel315@student.gsu.edu | Use Cases and Use Case Diagram/ Requirements | 3 and half hours | Split with Quang. | 2/23/26 | 9/10 |
| Quang Tran<br>(Coordinator) | qtran30@student.gsu.edu | Use Cases / Database Management | About 2 and half hours | Worked with Vrushti. Split with Kirtan. | 2/23/26 | 9/10 |
| Vishal Venu Bangrae | vbangrae1@student.gsu.edu | User Management Implementation | 3 hr | Worked with Quang. | 2/23/26 | 10/10 |

# Section 5

System Requirements (Use cases, Requirements, and use case diagrams)

## 5.1 Use Cases

**List of Use Case Name:**

1. User Management Feature
2. Link Bank Account and Import Transactions
3. View Dashboard and Financial Visualizations
4. Calculate Investment Readiness Score
5. Receive Educational Guidance
6. Interact with AI Financial Chatbot
7. Configure Alerts and Notifications
8. Run Investment Simulation
9. Monitor Transactions and Detect Anomalies
10. Manage Financial Goals and Growth Expectations

**Use Case no.:** 1

**Use Case Name:** User Management Feature

**Actors:** New User, Existing User, Admin, Auth Service (MFA), Database (MySQL)

**Description:** The use case begins when a new or existing user opens the application and selects either sign up or log in. The system collects required credentials and profile information, validates the data, and initiates multi-factor authentication through the authentication service. Upon successful verification, the system securely creates or updates the user record in the MySQL database and establishes an authenticated session. Additionally, administrators may manage user accounts by enforcing password resets, disabling accounts, or reviewing authentication audit logs to maintain platform integrity.

**Alternate Path:** The user may choose the forgot password option, prompting the system to send a reset link followed by identity verification. The user may also authenticate through a supported identity provider if enabled.

**Exception Path:** If credentials are invalid or MFA verification fails repeatedly, the system denies access, logs the attempt, and may temporarily lock the account for security protection.

**Pre-condition:** The user has network access and the authentication services are available.

**Post-condition:** The user is securely authenticated with an active session or blocked with a recorded audit trail.

**Use Case no.:** 2

**Use Case Name:** Link Bank Account and Import Transactions

**Actors:** User, Bank Aggregator API, Transaction Service, MySQL Database

**Description:** This use case starts when a user elects to connect a bank account to the platform. The system redirects the user through a secure bank aggregation flow where consent and authorization are granted. Once approved, the system receives encrypted access credentials, securely stores them, and retrieves transaction data through the transaction service. The imported transactions are normalized, categorized, and saved into the MySQL database, enabling spending analysis and downstream readiness scoring updates.

**Alternate Path:** The user may connect only selected accounts instead of linking multiple financial sources.

**Exception Path:** If authorization fails or tokens expire, the system prompts the user to retry authentication without storing incomplete data.

**Pre-condition:** The user is authenticated and possesses a valid bank account.

**Post-condition:** Transactions are securely stored and available for analysis.

**Use Case no.:** 3

**Use Case Name:** View Dashboard and Financial Visualizations

**Actors:** User, FlowFund System, MySQL Database

**Description:** This use case begins when a user navigates to the dashboard after authentication. The FlowFund system retrieves stored transaction data, goals, and historical analysis from the

database. The system processes this information and displays interactive charts and visualizations showing spending trends, income stability, cash buffer status, and readiness score history.

**Alternate Path:** The user may choose different time ranges or visualization types (weekly, monthly, category-based).

**Exception Path:** If data retrieval fails or incomplete data is available, the system displays a warning message and shows partial results.

**Pre-condition:** The user is authenticated and has at least one linked bank account.

**Post-condition:** The user views an updated financial dashboard with current analytics.

**Use Case no.:** 4

**Use Case Name:** Calculate Investment Readiness Score

**Actors:** FlowFund System, AI Readiness Engine, MySQL Database

**Description:** This use case starts when sufficient transaction and profile data is available. The FlowFund system sends spending behavior, income stability, budgeting habits, and risk tolerance data to the AI readiness engine. The engine computes an investment readiness score and returns the result to the system, which stores it in the database and makes it available for display.

**Alternate Path:** The system may recompute the score periodically or after major spending changes.

**Exception Path:** If the AI service is unavailable, the system retains the last valid readiness score and notifies the user.

**Pre-condition:** User transaction data and risk profile are available.

**Post-condition:** An updated investment readiness score is stored and accessible.

**Use Case no.:** 5

**Use Case Name:** Receive Educational Guidance

**Actors:** User, FlowFund System, AI Services

**Description:** When a user's readiness score is below the investment threshold, the system provides personalized educational guidance instead of investment recommendations. The FlowFund system sends contextual financial data to the AI service, which generates explainable guidance focused on budgeting, saving, and financial literacy.

**Alternate Path:** The user may request additional explanations or examples.

**Exception Path:** If AI services fail, the system displays static educational content.

**Pre-condition:** The user has a calculated readiness score.

**Post-condition:** The user receives tailored educational guidance.

**Use Case no.:** 6

**Use Case Name:** Interact with AI Financial Chatbot

**Actors:** User, AI Chatbot Service, FlowFund System

**Description:** This use case begins when a user opens the AI chatbot interface. The user submits financial questions, and the FlowFund system provides relevant financial context to the chatbot service. The AI generates guarded responses focused on education, explanations, and readiness insights.

**Alternate Path:** The user may ask follow-up questions within the same session.

**Exception Path:** If the chatbot service is unavailable, the system informs the user and disables the chatbot temporarily.

**Pre-condition:** The user is authenticated.

**Post-condition:** The user receives AI-generated financial guidance.

**Use Case no.:** 7

**Use Case Name:** Configure Alerts and Notifications

**Actors:** User, FlowFund System, Notification Service

**Description:** The user configures alert preferences such as spending spikes, low cash buffer warnings, readiness score changes, and goal milestones. The FlowFund system stores alert rules and monitors data continuously to trigger notifications when conditions are met.

**Alternate Path:** The user may disable or modify alerts at any time.

**Exception Path:** If notification delivery fails, the system retries or logs the failure.

**Pre-condition:** The user is authenticated and has transaction data.

**Post-condition:** Alerts are configured and actively monitored.

**Use Case no.:** 8

**Use Case Name:** Run Investment Simulation

**Actors:** User, FlowFund System, AI Simulation Engine

**Description:** This use case starts when the user selects an investment simulation option. The system collects goal data, time horizon, and risk tolerance, then sends this information to the AI simulation engine. The engine returns projected growth scenarios, which are displayed to the user.

**Alternate Path:** The user may adjust parameters and rerun the simulation.

**Exception Path:** If simulation services are unavailable, the system displays an error message.

**Pre-condition:** The user has defined goals and risk preferences.

**Post-condition:** Simulated investment projections are displayed.

**Use Case no.:** 9

**Use Case Name:** Monitor Transactions and Detect Anomalies

**Actors:** FlowFund System, Transaction Service, AI Services

**Description:** The FlowFund system continuously monitors incoming transaction data. AI services analyze spending patterns to detect anomalies or unusual spending behavior. Detected anomalies are logged and may trigger alerts.

**Alternate Path:** Minor anomalies may be logged without notifying the user.

**Exception Path:** If transaction data is delayed, monitoring resumes when data becomes available.

**Pre-condition:** Transaction monitoring is enabled

**Post-condition:** Anomalies are detected and processed appropriately.

**Use Case no.:** 10

**Use Case Name:** Manage Financial Goals and Growth Expectations

**Actors:** FlowFund System, MySQL Database

**Description:** This use case begins when the user creates or updates financial goals and growth expectations. The FlowFund system validates inputs, stores goal data, and uses it for readiness scoring and simulations.

**Alternate Path:** The user may edit or delete goals.

**Exception Path:** Invalid inputs result in validation errors.

**Pre-condition:** The user is authenticated.

**Post-condition:** Valid financial goals and growth expectations are saved and actively tracked.

## Section 5.2 (User Case Requirements)

**Requirement number:** 1

**Use Case number:** 1

**Introduction:** This is an easy and secure process for users to register and log in to access the dashboard and financial tools.

**Inputs:**

1. Email / Username
2. Password
3. Multi-factor authentication code

**Requirements Description:** Requires all fields to be filled, and credentials should be validated against the MySQL database. MFA should also be verified, as a failed login must be logged and may result in a temporary account lock. Admin can also reset the password or disable accounts.

**Outputs:**

1. User login successful
2. Access was denied with an error message, and an audit log was recorded.

**Requirement number:** 2

**Use Case number:** 2 (Link Bank Account and Import Transactions)

**Introduction:** This requirement enables users to securely connect bank accounts and import transaction data for financial analysis

**Inputs:**

1. User consent
2. Bank authorization tokens
3. Transaction records

**Requirements Description:** The system should integrate with a bank aggregation API to authenticate user consent, retrieve transaction data securely, normalize and categorize transactions, and store them in the database. No incomplete or unauthorized data shall be stored.

**Outputs:** Successfully imported and stored transactions

**Requirement number:** 3

**Use Case number:** 3 (View Dashboard and Financial Visualizations)

**Introduction:** This requirement provides users with visual insights into their financial data.

**Inputs:**

1. Transaction history
2. Goals data
3. Readiness score history

**Requirements Description:** The system should generate dashboards with charts and analytics showing spending trends, income stability, cash buffers, and historical readiness scores. Users shall be able to adjust time ranges and views.

**Outputs:** Interactive financial dashboard

**Requirement number:** 4

**Use Case number:** 4 (Calculate Investment Readiness Score)

**Introduction:** This requirement defines the AI-based evaluation of user investment readiness.

**Inputs:**

1. Spending behavior data

2. Income stability metrics

3. Risk tolerance profile

**Requirements Description:** The system should compute an investment readiness score using AI models based on financial behavior and risk tolerance. The score shall be recalculated when new data is available and stored for historical tracking.

**Outputs:** Updated investment readiness score

**Requirement number:** 5

**Use Case number:** 5 (Receive Educational Guidance)

**Introduction:** This requirement ensures users receive guidance when they are not ready to invest.

**Inputs:**

1. Readiness score

2. Financial behavior context

**Requirements Description:** When readiness is below a defined threshold, the system shall provide AI-generated educational guidance instead of investment recommendations. Guidance must be explainable, safe, and non-actionable.

**Outputs:** Personalized educational content

**Requirement number:** 6

**Use Case number:** and 6 (Interact with AI Financial Chatbot)

**Introduction:** The requirement supports financial guidance through an AI chatbot

**Inputs:**

1. User queries
2. Financial context

**Requirements Description:** The system should provide an AI chatbot that answers financial questions. The chatbot should not perform trading on its own or give unsafe financial advice.

**Outputs:** AI-generated chatbot responses

**Requirement number:** 7

**Use Case number:** 7 (Configure Alerts and Notifications)

**Introduction:** and This requirement allows users to receive real-time financial alerts.

**Inputs:**

1. Alert preferences
2. Transaction and readiness data

**Requirements Description:** The system should allow users to configure alerts for spending spikes, low cash buffers, readiness score changes, and goal milestones. Alerts shall be triggered automatically when conditions are met.

**Outputs:** Delivered notifications and alerts

**Requirement number:** 8

**Use Case number:** 8 (Run Investment Simulation)

**Introduction:** This requirement enables users to explore hypothetical investment outcomes.

**Inputs:**

1. Financial goals
2. Time horizon
3. Risk tolerance

**Requirements Description:** The system should generate investment growth simulations based on user-defined parameters. Simulations must be informational only and must not execute real transactions.

**Outputs:** Investment projection results

**Requirement number:** 9

**Use Case number:** 9 (Monitor Transactions and Detect Anomalies)

**Introduction:** This requirement ensures continuous monitoring for unusual financial activity.

**Inputs:**

1. Incoming transaction data

**Requirements Description:**The system should monitor transactions using AI-based anomaly detection algorithms. A transaction shall be flagged as anomalous if it exceeds predefined thresholds such as:a transaction amount exceeding the user's historical spending average by a defined percentage (e.g., 200%)

Detected anomalies shall be logged and may trigger alerts depending on configured user preferences.

**Outputs:** Logged anomaly events and triggered alerts when threshold conditions are met.

**Requirement number:** 10

**Use Case number:** 10 (Manage Financial Goals and Growth Expectations)

**Introduction:** This requirement allows users to define and manage financial goals.

**Inputs:**

1. Goal amount
2. Time horizon
3. Growth expectations

**Requirements Description:** The system should allow users to create, update, and delete financial goals. These goals shall be used in readiness scoring, simulations, and progress tracking.

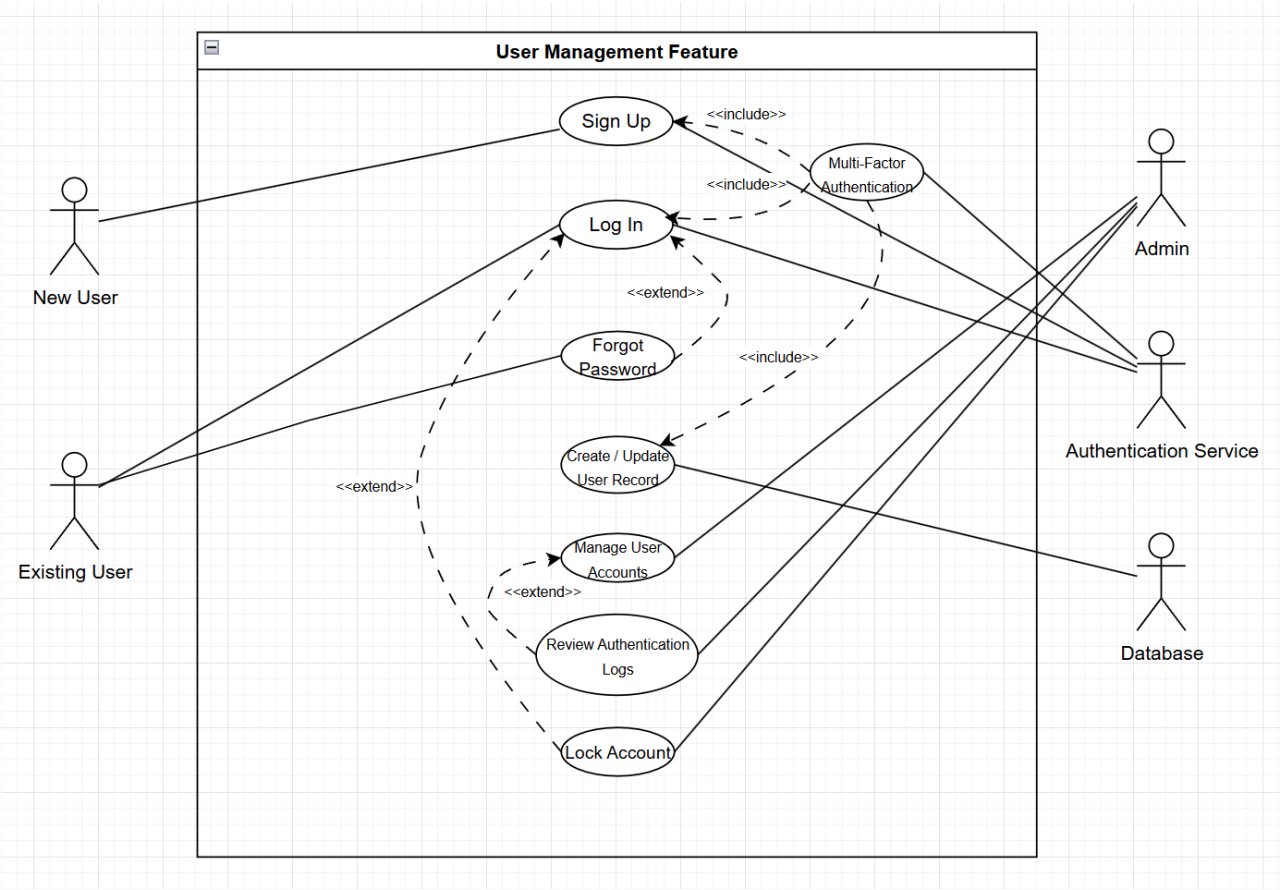
**Outputs:** Saved and active financial goals

## Section 5.3 (Use Case Diagram)

**Use Case no.:** 1

**Use Case Diagram no: 1**

**Use Case Name:** User Management Feature

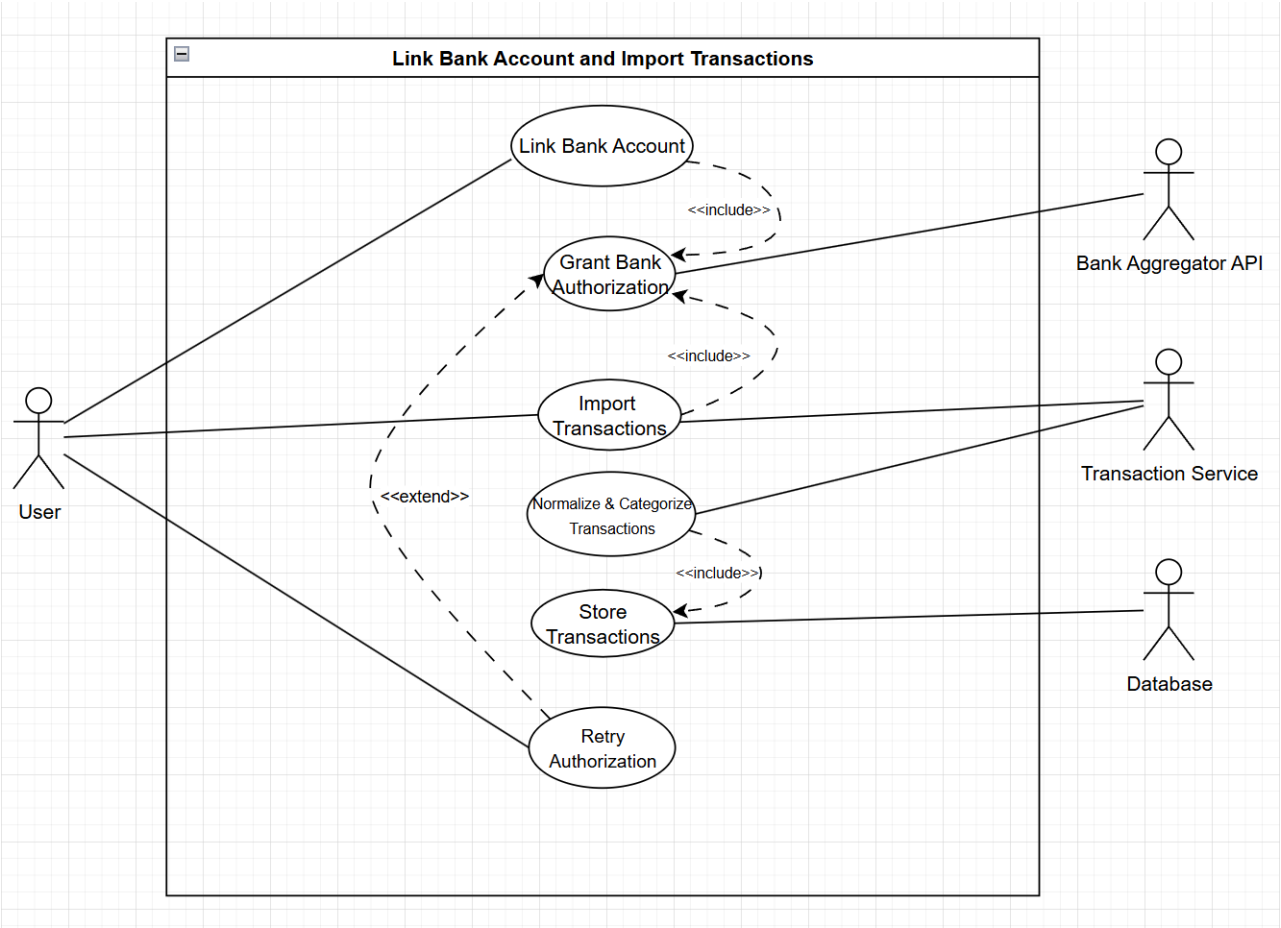



https://drive.google.com/file/d/1840jSecUjLyQ-iFu7dp8CqB4k-WURv-P/view?usp=sharing

**Use Case no.:** 2

**Use Case Diagram no: 2**

**Use Case Name:** Link Bank Account and Import Transactions

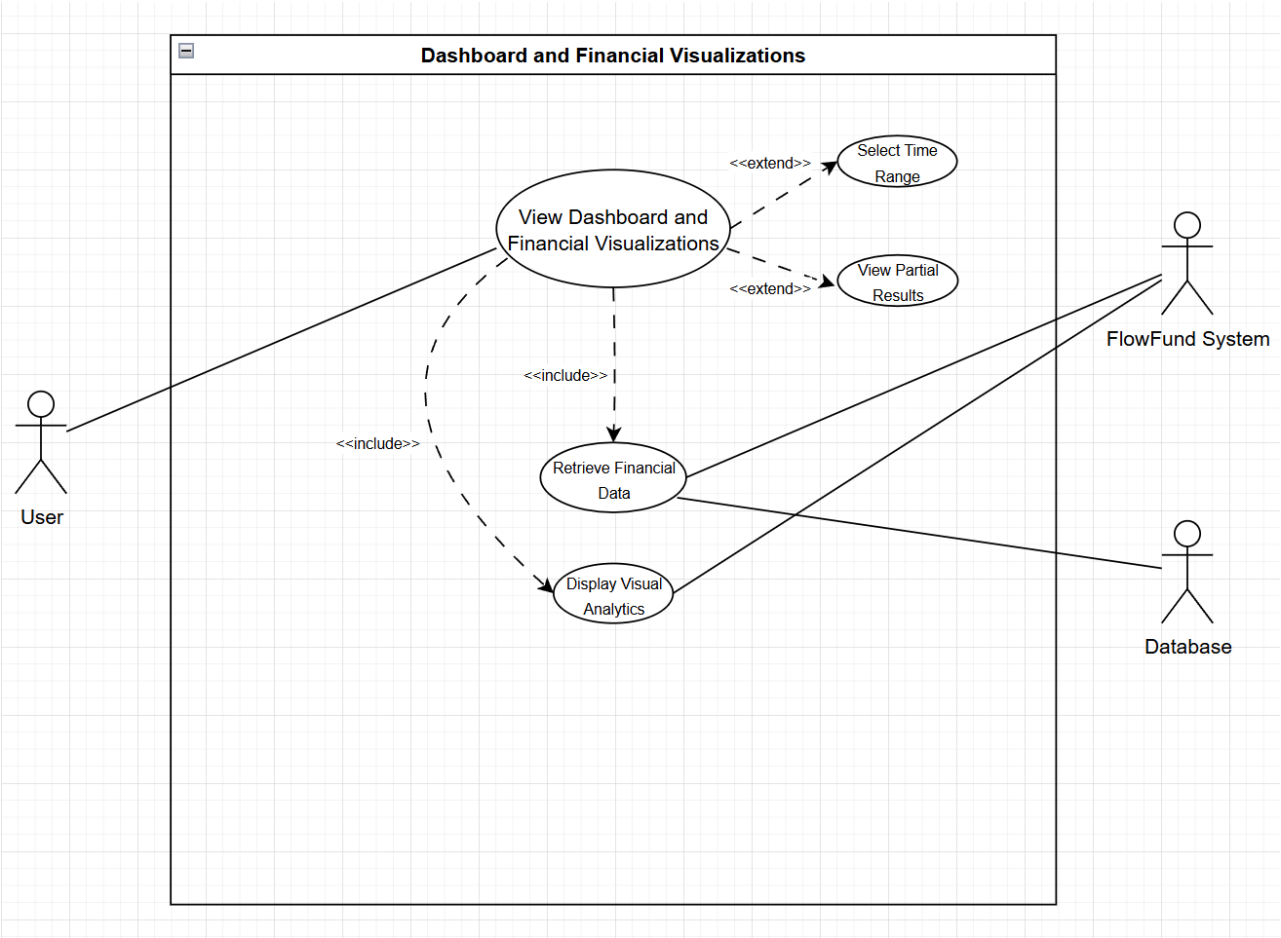



https://drive.google.com/file/d/1OeTsIvsCQWKoaJRcpU8D8U9k4qDt0FWn/view?usp=sharing

**Use Case no.:** 3

**Use Case Diagram no: 3**

**Use Case Name:** Dashboard and Financial Visualizations

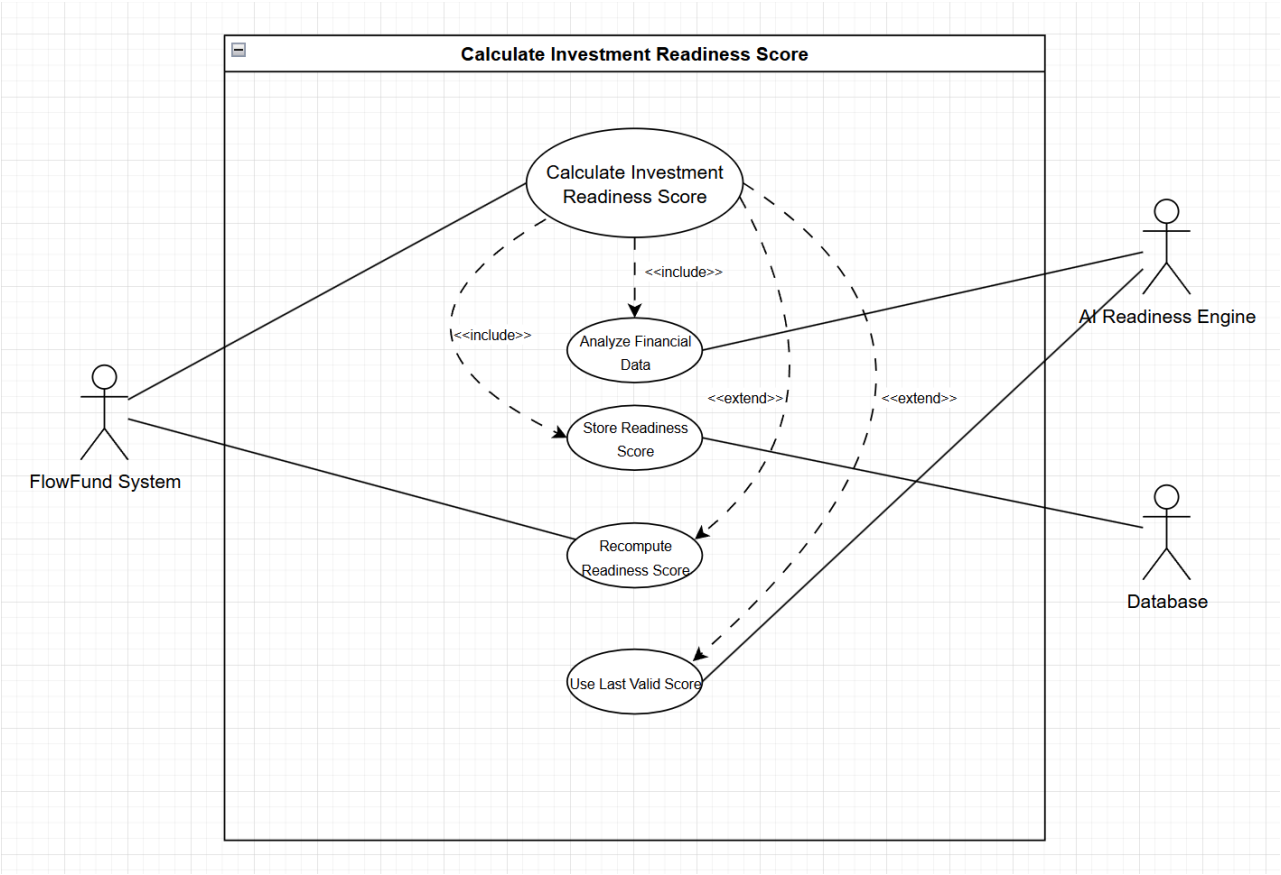



https://drive.google.com/file/d/1oxUtjVLlTx9hYv_rbLwkozIh_tn9OfCS/view?usp=sharing

**Use Case no.:** 4

**Use Case Diagram no: 4**

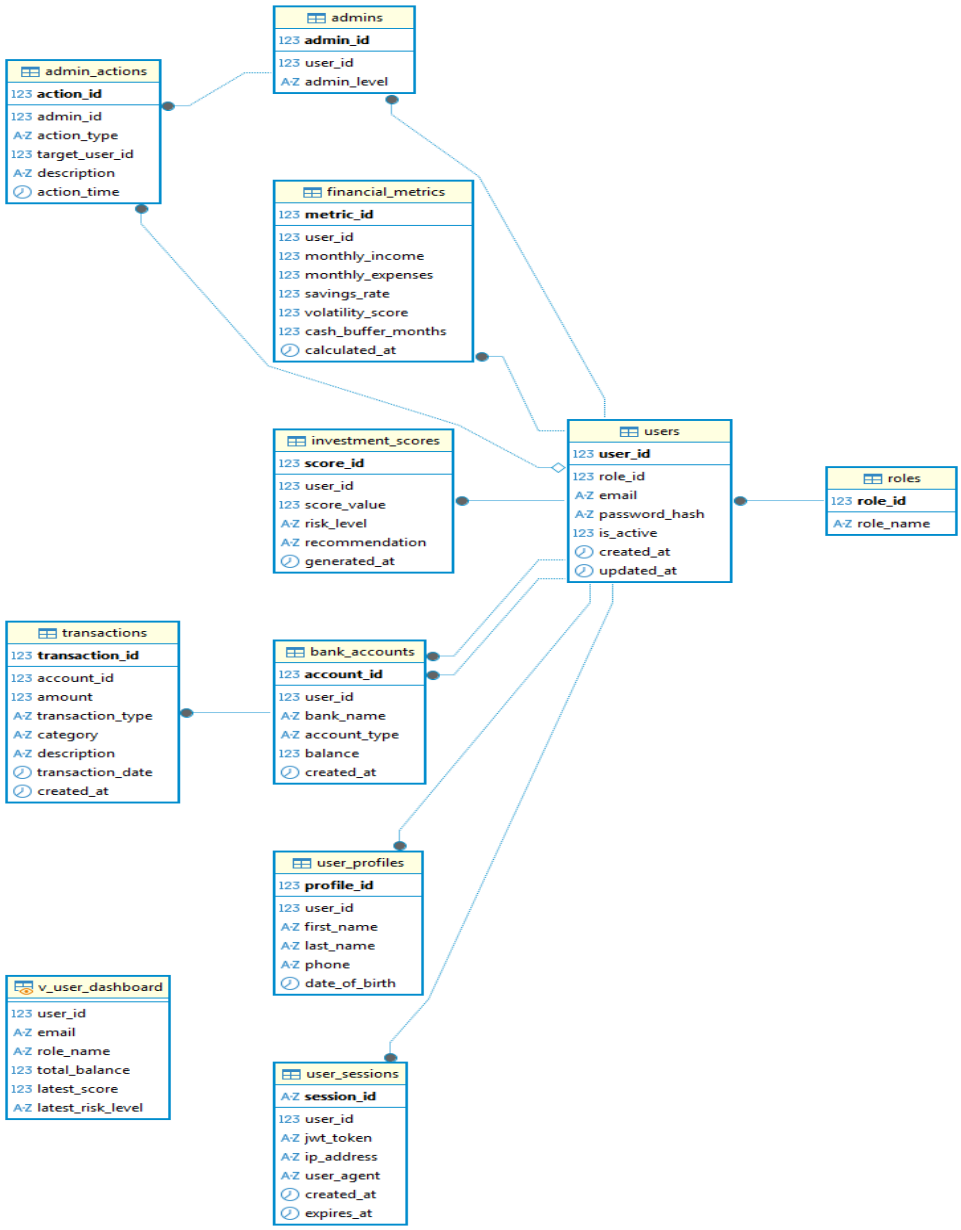
**Use Case Name:** Calculate Investment Readiness Score




https://drive.google.com/file/d/1VUl2FqawXPP6F3_te6BR8CUVlBpvNfSj/view?usp=sharing

A use case diagram involves illustrating the interactions between actors and the system's use cases, with actors represented as stick figures and use cases in ovals. The system boundary is represented within a rectangle, and arrows and lines are used to show interactions between actors, interactions between an actor and a system, and interactions between systems. Additionally, this diagram includes 'includes' and 'excludes' relationships, each associated with the name of the respective use case.

## Section 6: Database management




FundFlow AI uses MySQL as the database management system (DBMS). The system is implemented as a relational database with enforced integrity through Primary Keys and Foreign Keys as shown in the ER diagram.

1) **roles**

Purpose: Role-based access control (USER, ADMIN).
Attributes (with types):

- **role_id INT — PK, AUTO_INCREMENT**
- **role_name VARCHAR(50) — UNIQUE, NOT NULL**

Relationships:

- **roles.role_id (PK) → users.role_id (FK)**
    - 1 role : many users

**2) users**

**Purpose:** Core user authentication/identity table.
**Attributes (with types):**

- user_id **INT** — **PK**, AUTO_INCREMENT
- role_id **INT** — **FK** → roles(role_id), NOT NULL
- email **VARCHAR(150)** — UNIQUE, NOT NULL
- password_hash **VARCHAR(255)** — NOT NULL
- is_active **BOOLEAN** — DEFAULT TRUE
- created_at **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP
- updated_at **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

**Relationships (users as parent):**

- users.user_id (**PK**) → bank_accounts.user_id (**FK**) (**1 : many**)
- users.user_id (**PK**) → transactions (indirect via bank_accounts)
- users.user_id (**PK**) → financial_metrics.user_id (**FK**) (**1 : many**)

- users.user_id (**PK**) → investment_scores.user_id (**FK**) (**1 : many**)
- users.user_id (**PK**) → user_profiles.user_id (**FK**) (**1 : 1 / 1 : many** depending on enforcement)
- users.user_id (**PK**) → user_sessions.user_id (**FK**) (**1 : many**)
- users.user_id (**PK**) → admins.user_id (**FK**) (**1 : 0..1**)

**3) admins**

**Purpose:** Admin-specific data for governance/management accounts.
**Attributes (with types):**

- admin_id **INT** — **PK**, AUTO_INCREMENT
- user_id **INT** — **FK** → users(user_id), NOT NULL
- admin_level **ENUM('SUPER_ADMIN','MODERATOR')** — DEFAULT 'MODERATOR'

**Relationships:**

- admins.admin_id (**PK**) → admin_actions.admin_id (**FK**)
    - **1 admin : many admin actions**

**4) admin_actions**

**Purpose:** Audit logging for admin activity (compliance & security).
**Attributes (with types):**

- action_id **INT** — **PK**, AUTO_INCREMENT
- admin_id **INT** — **FK** → admins(admin_id), NOT NULL
- action_type **VARCHAR(100)**
- target_user_id **INT** — **FK** → users(user_id)
- description **TEXT**
- action_time **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP

**Relationships:**

- admin_actions.admin_id (**FK**) → admins.admin_id (**PK**)
- admin_actions.target_user_id (**FK**) → users.user_id (**PK**)

**5) user_profiles**

**Purpose:** Stores personal profile information separate from authentication.
**Attributes (with types):**

- profile_id **INT** — **PK**, AUTO_INCREMENT
- user_id **INT** — **FK** → users(user_id), NOT NULL
- first_name **VARCHAR(100)**
- last_name **VARCHAR(100)**
- phone **VARCHAR(20)**
- date_of_birth **DATE**

**Relationships:**

- user_profiles.user_id (**FK**) → users.user_id (**PK**)

**6) user_sessions**

**Purpose:** Session tracking (JWT/session security, expiration).
**Attributes (with types):**

- session_id **VARCHAR(255)** — **PK**
- user_id **INT** — **FK** → users(user_id), NOT NULL
- jwt_token **TEXT** — NOT NULL
- ip_address **VARCHAR(45)**
- user_agent **TEXT**

- created_at **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP
- expires_at **TIMESTAMP**

**Relationships:**

- user_sessions.user_id (**FK**) → users.user_id (**PK**)

**7) bank_accounts**

**Purpose:** User-linked financial accounts (checking/savings/credit).
**Attributes (with types):**

- account_id **INT** — **PK**, AUTO_INCREMENT
- user_id **INT** — **FK** → users(user_id), NOT NULL
- bank_name **VARCHAR(150)**
- account_type **ENUM('CHECKING','SAVINGS','CREDIT')**
- balance **DECIMAL(15,2)** — DEFAULT 0.00
- created_at **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP

**Relationships:**

- bank_accounts.user_id (**FK**) → users.user_id (**PK**) (**1 user : many accounts**)
- transactions.account_id (**FK**) → bank_accounts.account_id (**PK**) (**1 account : many transactions**)

**8) transactions**

**Purpose:** Income/expense transaction history used for monitoring + analytics.
**Attributes (with types):**

- transaction_id **INT** — **PK**, AUTO_INCREMENT
- account_id **INT** — **FK** → bank_accounts(account_id), NOT NULL

- amount **DECIMAL(15,2)** — NOT NULL
- transaction_type **ENUM('INCOME','EXPENSE')**
- category **VARCHAR(100)**
- description **TEXT**
- transaction_date **DATE**
- created_at **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP

**Relationships:**

- transactions.account_id (**FK**) → bank_accounts.account_id (**PK**)

**9) financial_metrics**

**Purpose:** Derived monthly/periodic financial summaries used by the readiness engine.
**Attributes (with types):**

- metric_id **INT** — **PK**, AUTO_INCREMENT
- user_id **INT** — **FK** → users(user_id), NOT NULL
- monthly_income **DECIMAL(15,2)**
- monthly_expenses **DECIMAL(15,2)**
- savings_rate **DECIMAL(5,2)**
- volatility_score **DECIMAL(5,2)**
- cash_buffer_months **DECIMAL(5,2)**
- calculated_at **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP

**Relationships:**

- financial_metrics.user_id (**FK**) → users.user_id (**PK**)

**10) investment_scores**

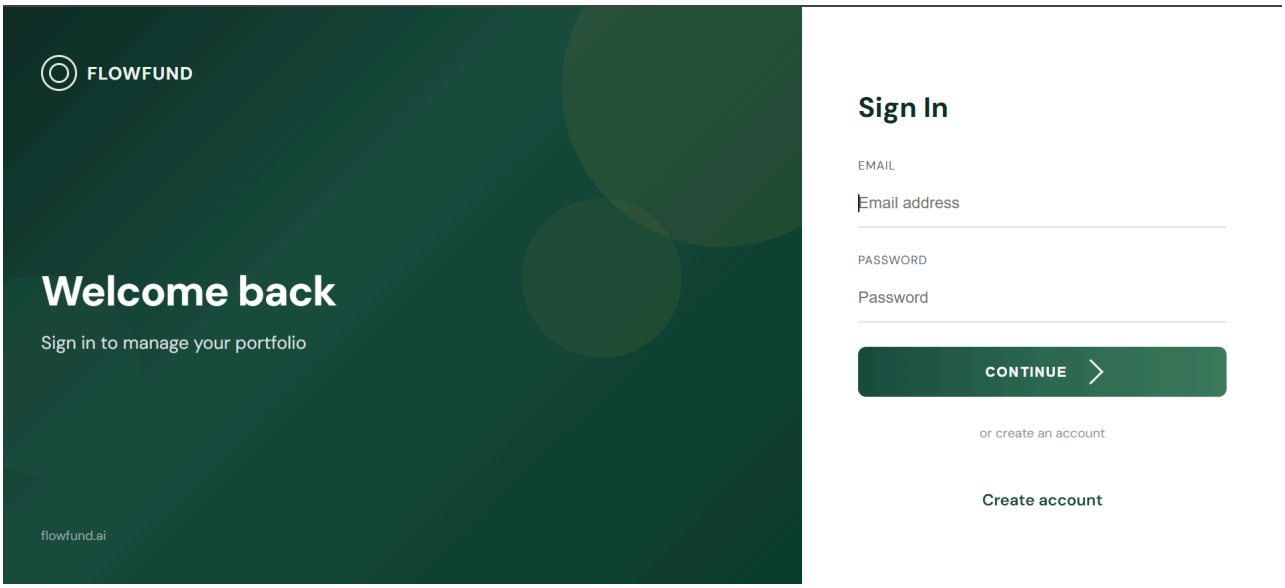
**Purpose:** Stores the investment readiness output (score + risk + explanation).
**Attributes (with types):**

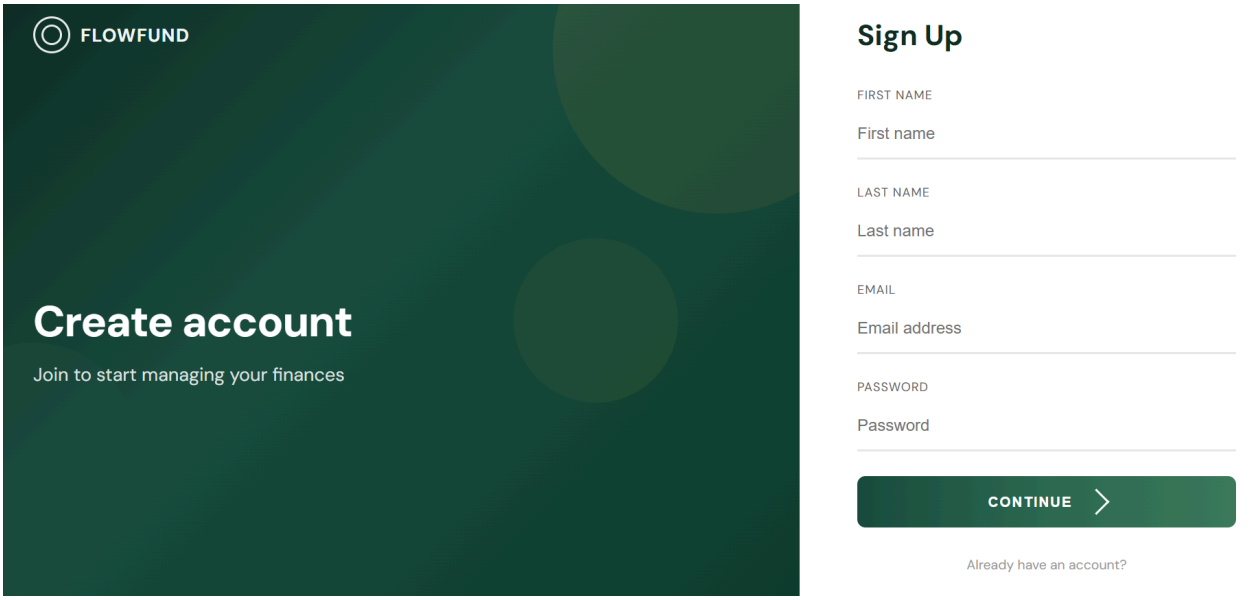
- score_id **INT** — **PK**, AUTO_INCREMENT
- user_id **INT** — **FK** → users(user_id), NOT NULL
- score_value **INT**
- risk_level **ENUM('LOW','MEDIUM','HIGH')**
- recommendation **TEXT**
- generated_at **TIMESTAMP** — DEFAULT CURRENT_TIMESTAMP

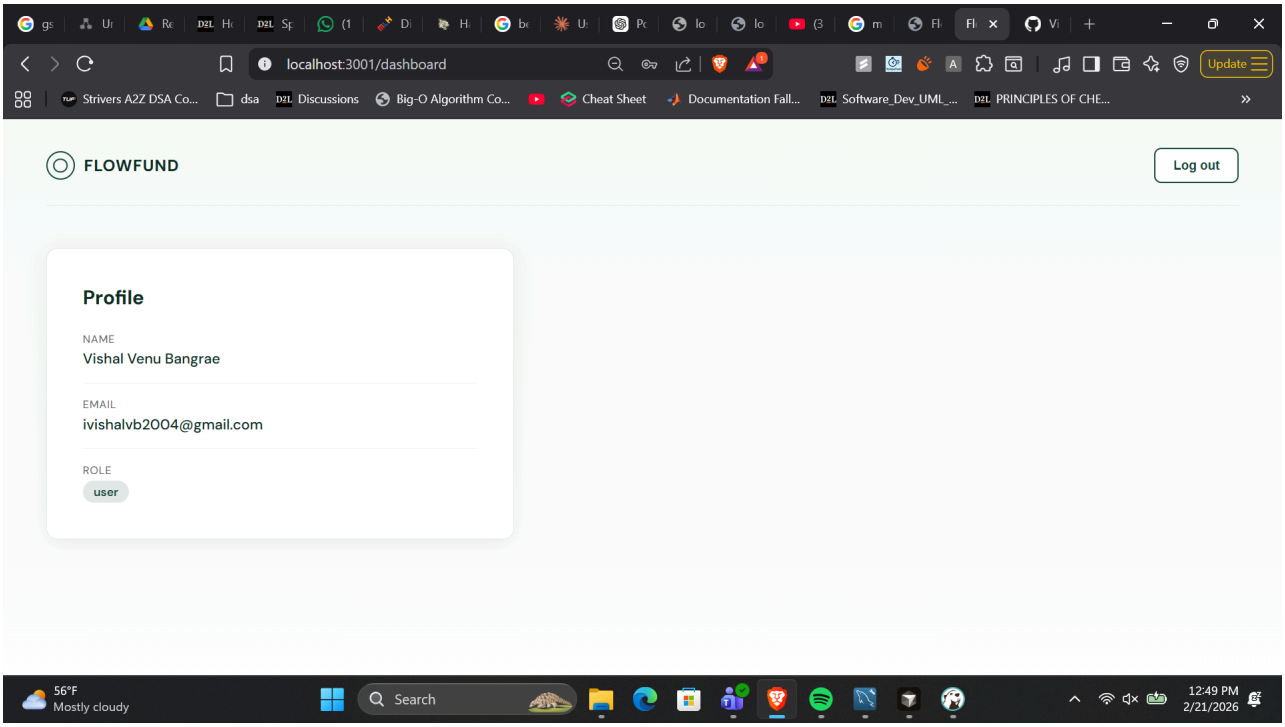
**Relationships:**

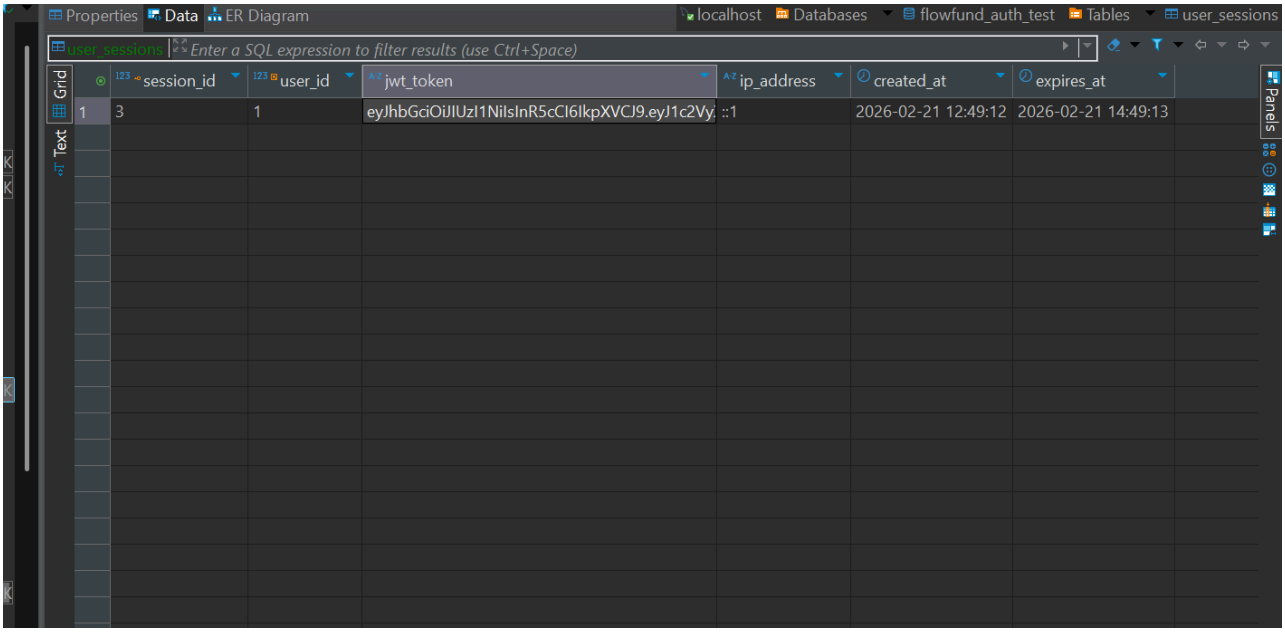
- investment_scores.user_id (**FK**) → users.user_id (**PK**)

## Section 7: User Management Implementation




FLOWFUND
Create account
Join to start managing your finances
Sign Up
FIRST NAME
First name
LAST NAME
Last name
EMAIL
Email address
PASSWORD
Password
CONTINUE
Already have an account?


localhost:3001/dashboard
Update
FLOWFUND
Log out
Profile
NAME
Vishal Venu Bangrae
EMAIL
ivishalvb2004@gmail.com
ROLE
user
56°F
Mostly cloudy
Search
12:49 PM
2/21/2026



| | session_id | user_id | jwt_token | ip_address | created_at | expires_at |
|---|---|---|---|---|---|---|
| 1 | 3 | 1 | eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2Vy | ::1 | 2026-02-21 12:49:12 | 2026-02-21 14:49:13 |

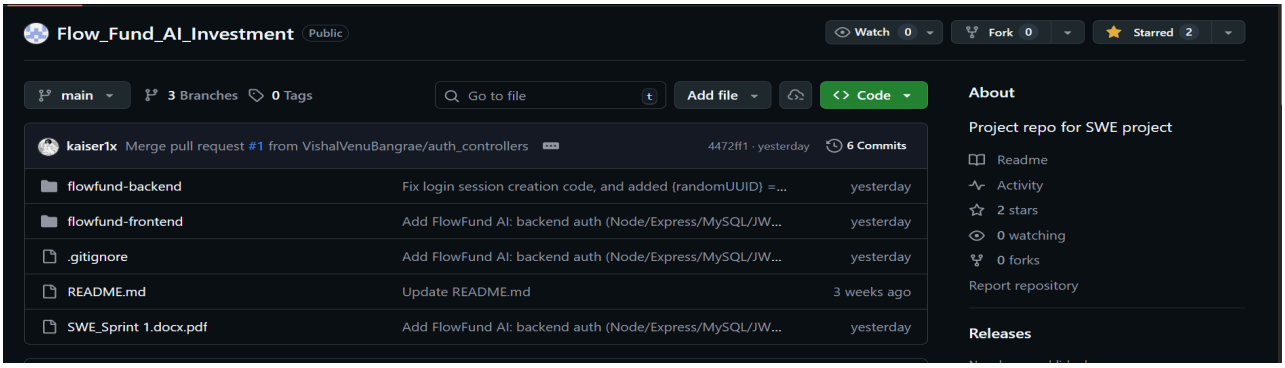
How it works

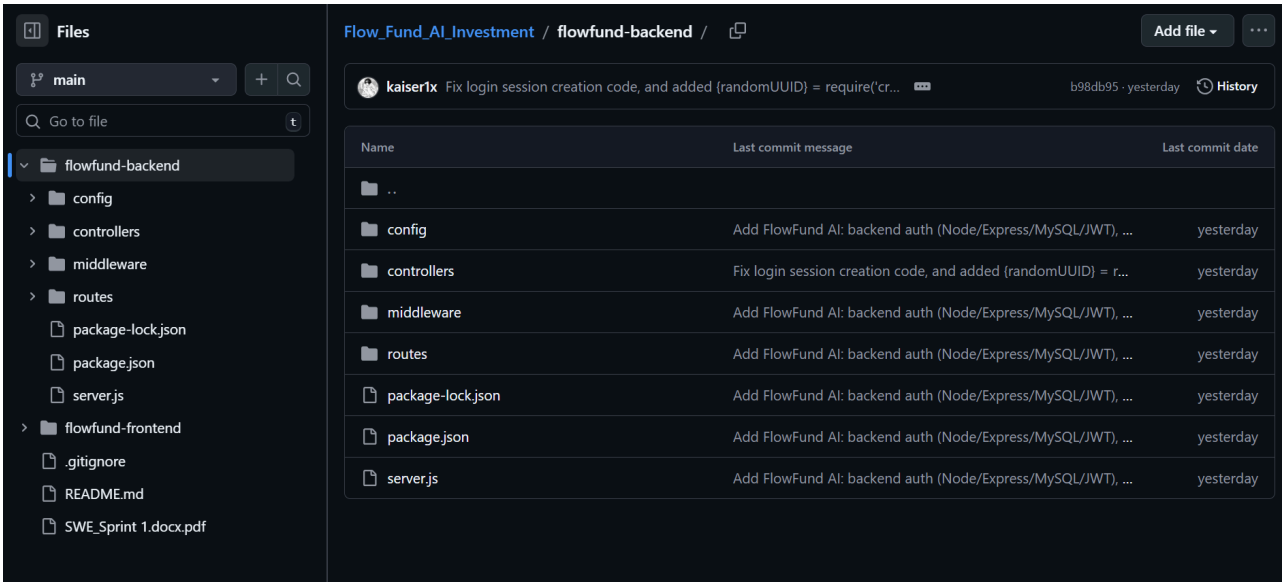
- User signs up or logs in; backend hashes passwords, issues a 2-hour JWT, and stores the session in user_sessions.
- The frontend sends the JWT as a Bearer token on each request; the backend checks the token and user_sessions before allowing access to /api/auth/profile.
- Logout deletes the session in the DB and clears the token; the app also logs out when the JWT expires (by time or on 401).

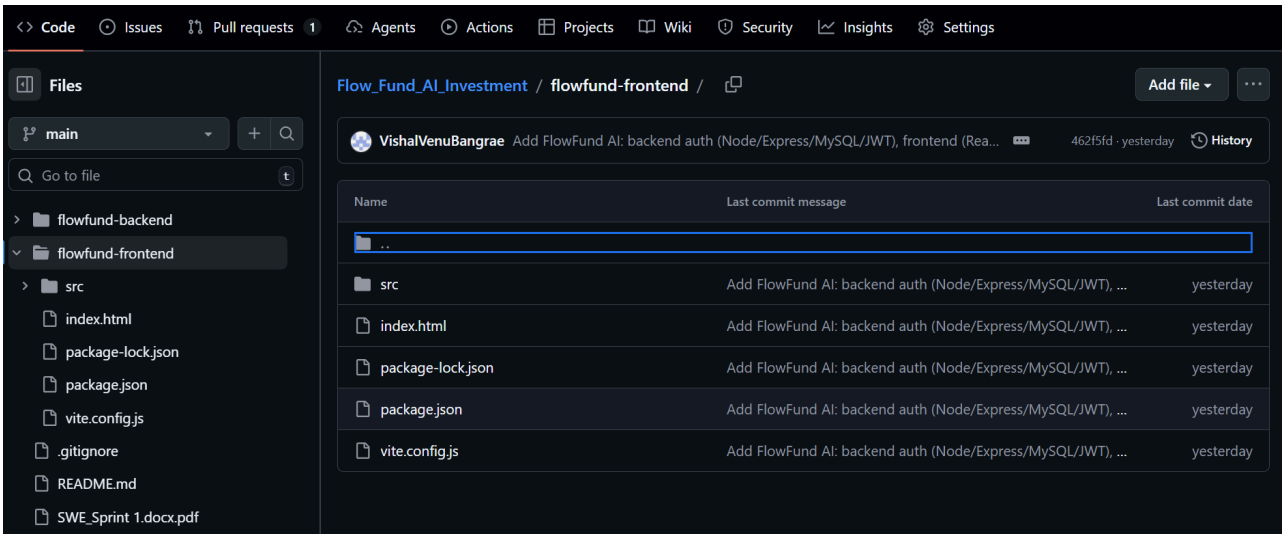
Github Details:

Github link: [https://github.com/VishalVenuBangrae/Flow_Fund_AI_Investment.git](https://github.com/VishalVenuBangrae/Flow_Fund_AI_Investment.git)

Github Screenshots:



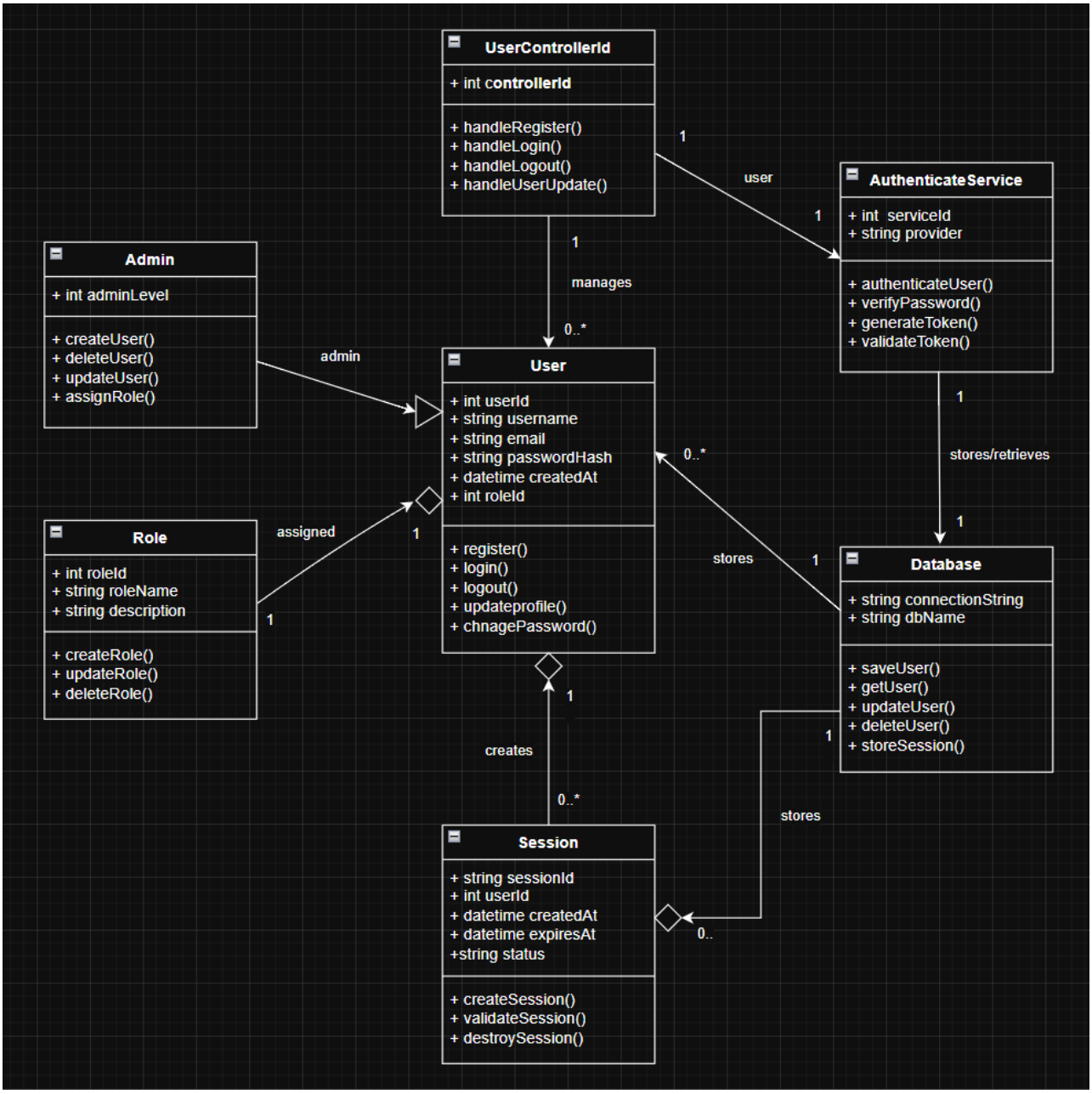

Files
main
Go to file
flowfund-backend
config
controllers
middleware
routes
package-lock.json
package.json
server.js
flowfund-frontend
.gitignore
README.md
SWE_Sprint 1.docx.pdf
Flow_Fund_AI_Investment / flowfund-backend /
Add file
kaiser1x Fix login session creation code, and added {randomUUID} = require('cr...
b98db95 · yesterday
History
Name
Last commit message
Last commit date
..
config
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
controllers
Fix login session creation code, and added {randomUUID} = r...
yesterday
middleware
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
routes
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
package-lock.json
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
package.json
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
server.js
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday


Code
Issues
Pull requests 1
Agents
Actions
Projects
Wiki
Security
Insights
Settings
Files
main
Go to file
flowfund-backend
flowfund-frontend
src
index.html
package-lock.json
package.json
vite.config.js
.gitignore
README.md
SWE_Sprint 1.docx.pdf
Flow_Fund_AI_Investment / flowfund-frontend /
Add file
VishalVenuBangrae Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), frontend (Rea...
462f5fd · yesterday
History
Name
Last commit message
Last commit date
..
src
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
index.html
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
package-lock.json
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
package.json
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday
vite.config.js
Add FlowFund AI: backend auth (Node/Express/MySQL/JWT), ...
yesterday

| Assignee Name | Email address | Task | Duration (hours) | Dependency | Due Date | Evaluation |
|---|---|---|---|---|---|---|
| Ayan Lakhani | alakhani18@student.gsu.edu | Diagrams and Diagram explanations | 1 hr | Worked with Vrushti | 3/9/26 | 8/10 |
| Vrushti Patel | vpatel131@student.gsu.edu | Class Diagram/ Sequence Diagram | 1 hr | Worked with Ayan and Kirtan | 3/9/26 | 9/10 |
| Kirtan Patel | kpatel315@student.gsu.edu | Improved Requirements/ Class Diagram | 1 hr | Worked with Vrushti | 3/9/26 | 10/10 |
| Quang Tran | qtran30@student.gsu.edu | Integrated frontend and backend, setting up the server via Vercel | 3 hr | Vishal | 3/9/26 | 8/10 |
| Vishal Venu Bangrae (Coordinator) | vbangrae1@student.gsu.edu | Updating backend and frontend, implementing table design, and hosting database on Railway | 2hr | Quang | 3/9/26 | 9/10 |

## Class Diagram (object Modeling, or structural modeling):

This class diagram represents a **user authentication and management system**. The **User** class is the main part of the system and stores user information such as username, email, password, and role. Users can perform actions like registering, logging in, logging out, updating their profile, and changing their password. The **Role** class defines the different roles a user can have, such as admin or regular user. The **Session** class manages login sessions and keeps track of when a session starts, expires, and whether it is active. This allows the system to manage multiple user logins. The **AuthService** class handles authentication by verifying passwords, generating tokens, and validating user sessions. The **Database** class stores and retrieves user and session data from the system. Finally, the **UserController** manages user requests like login or registration and communicates with the authentication service and database. The **Admin** class allows administrators to manage users by creating, updating, deleting users, and assigning roles.

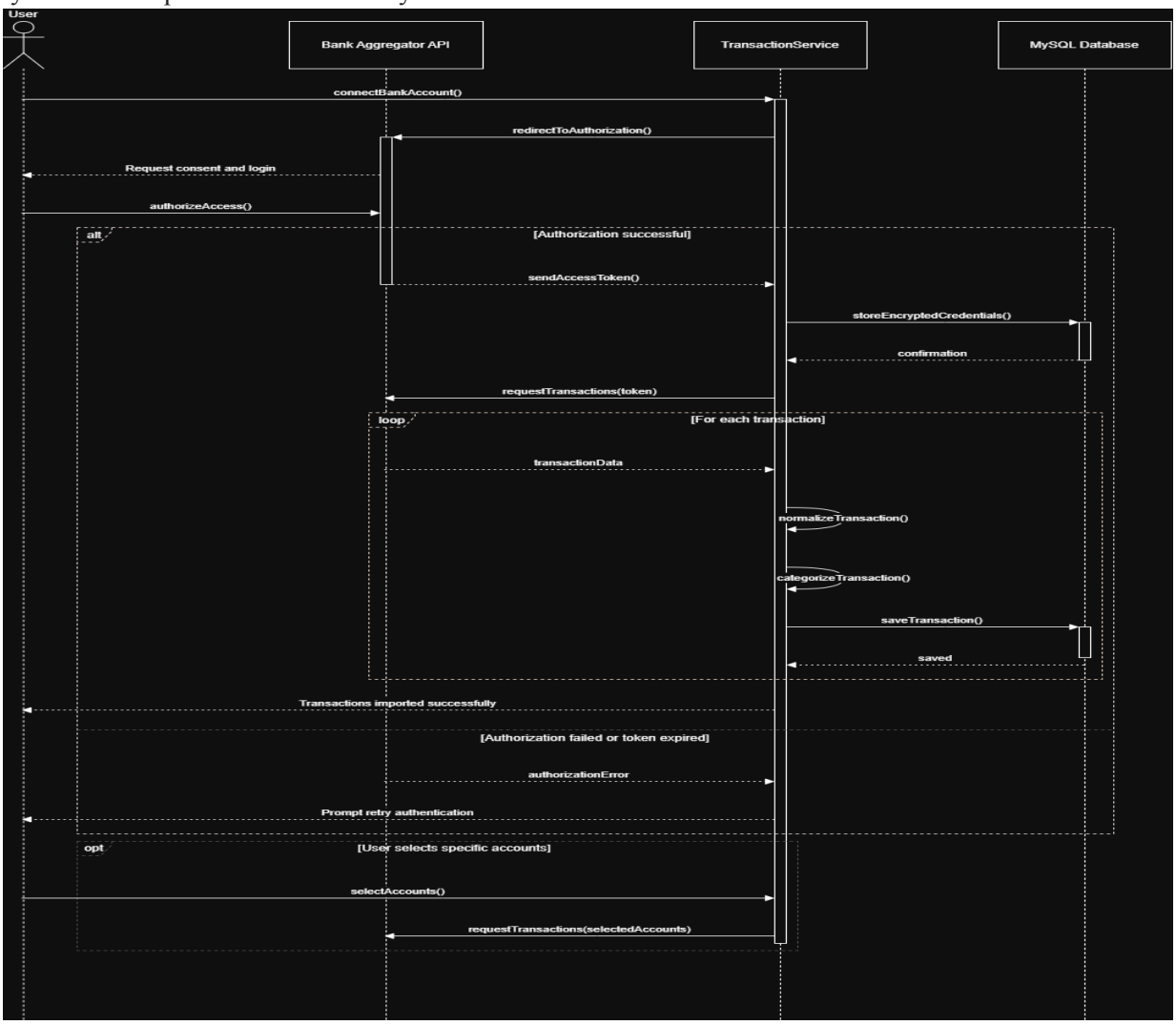



https://drive.google.com/file/d/1a3Jp32CClsqgWoAfpkhAQ7sdvqnaxOmA/view?usp=sharing

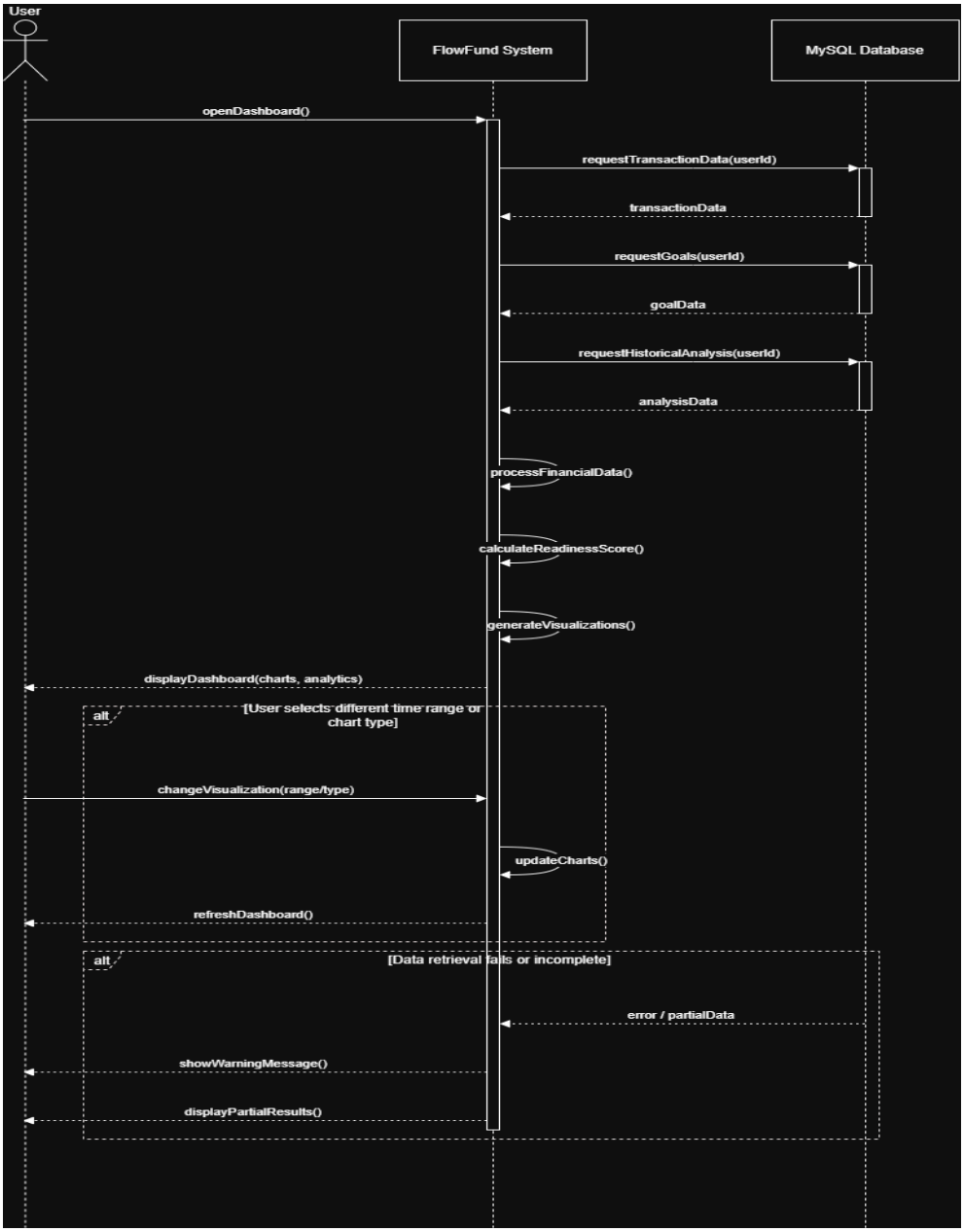
## Section 8:

**Use Case 2:** This sequence diagram shows the process of connecting a user's bank account and importing transactions into the system. The process begins when the User initiates a request to connect their bank account. The system then communicates with the Bank Aggregator API, which redirects the user to authorize access to their bank account. Once the user gives permission and the authorization is successful, the Bank Aggregator API sends an access token to the TransactionService. The service securely stores the

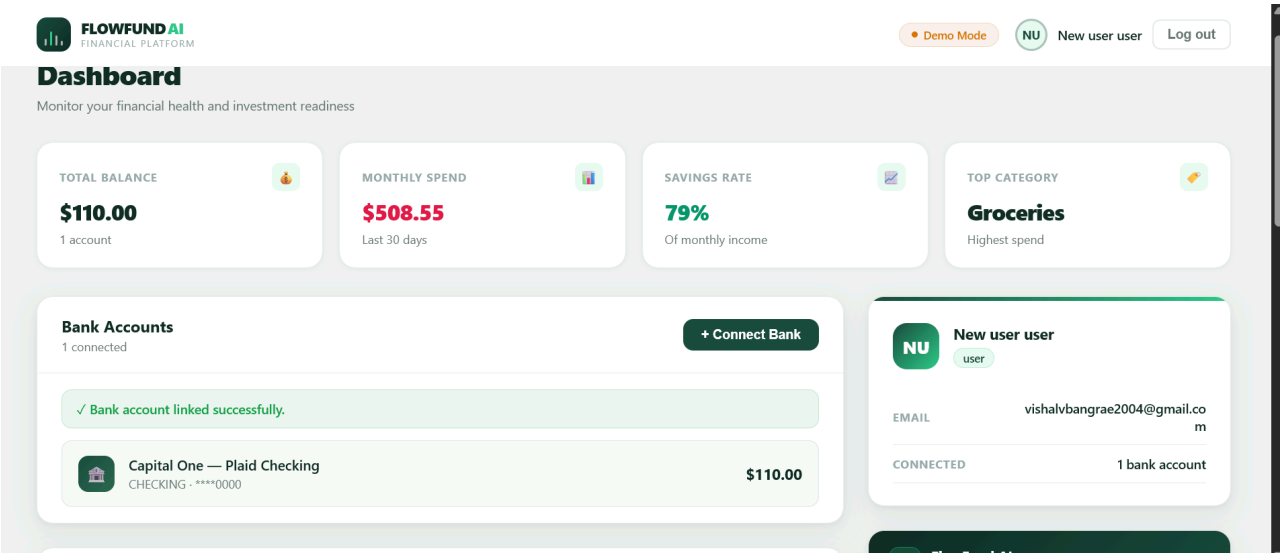
encrypted credentials in the MySQL Database and receives confirmation that the information has been saved. After authentication is complete, the TransactionService requests transaction data from the bank API using the access token. For each transaction received, the service normalizes the data, categorizes the transaction, and then stores it in the database. This loop continues until all transactions have been processed and saved. If the authorization fails or the access token expires, the system returns an authorization error and prompts the user to retry authentication. Optionally, the user can also select specific accounts, and the system will request transactions only for those selected accounts.

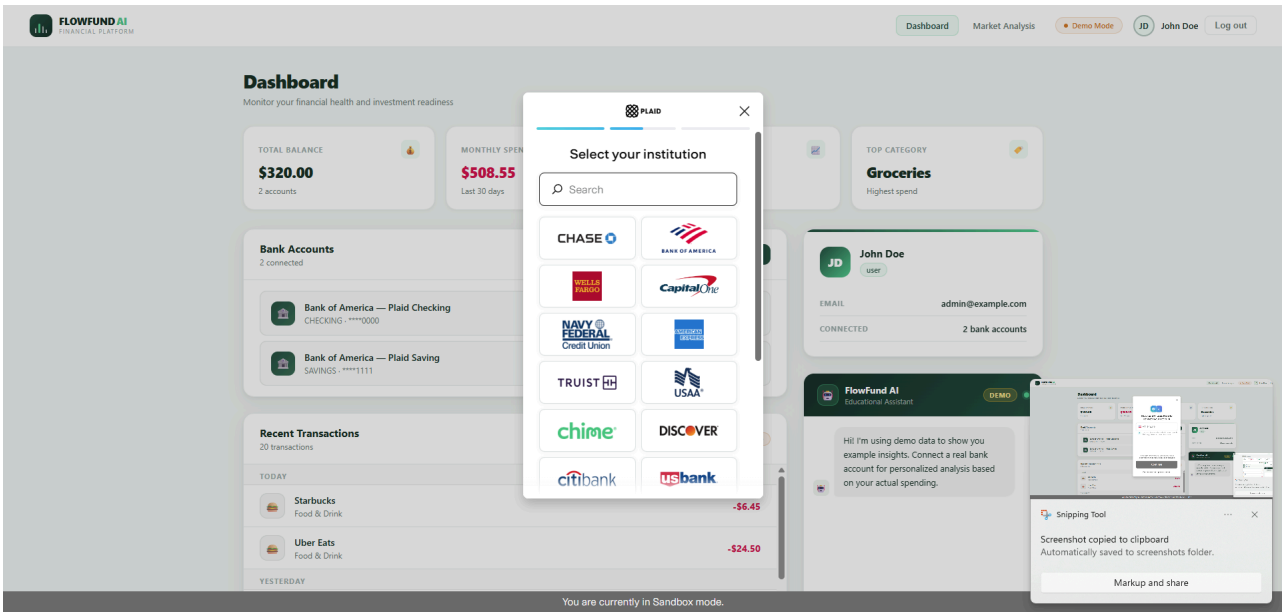


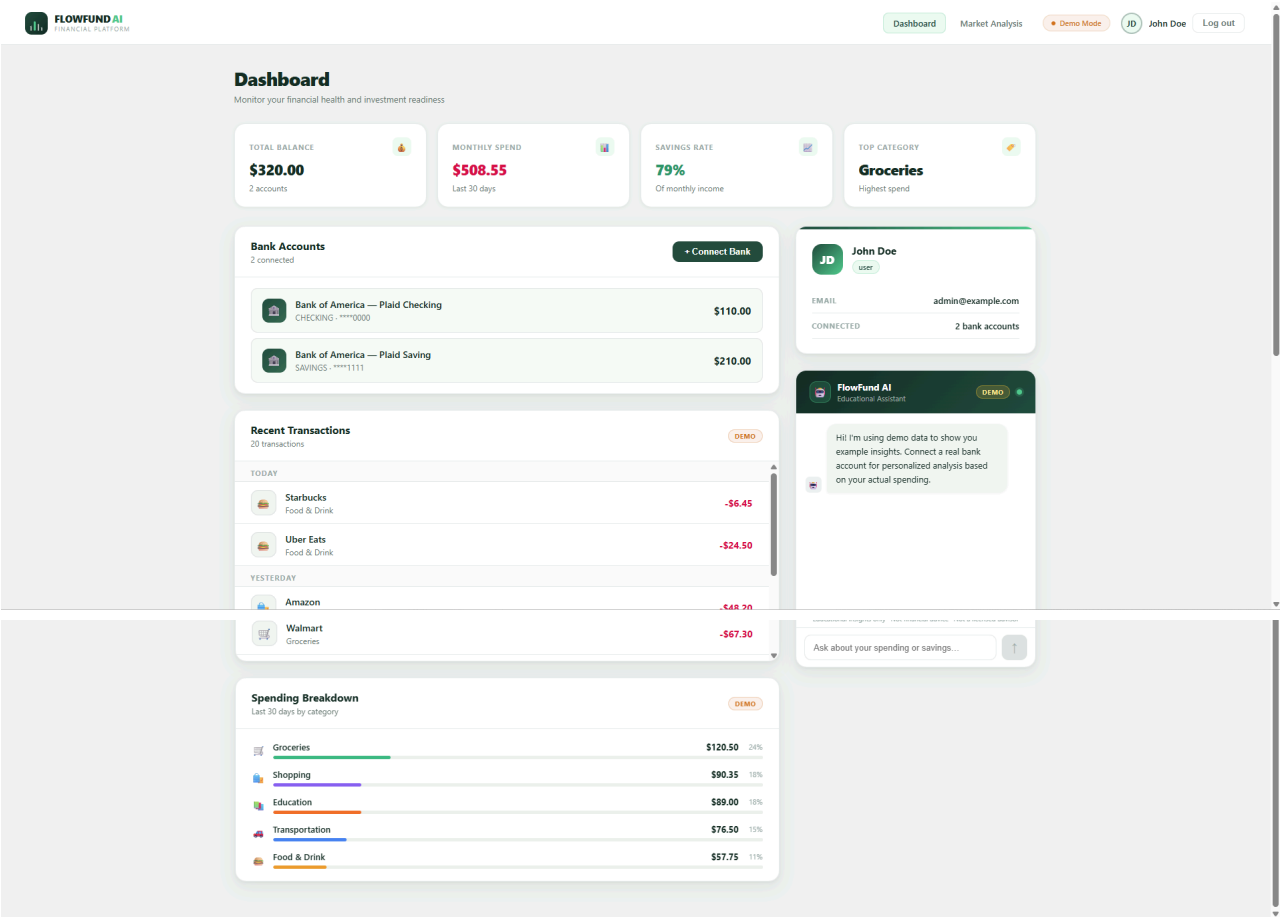

**Use Case 3:** This sequence diagram shows how a user views their financial dashboard in the FlowFund System. The process starts when the User opens the dashboard. The FlowFund System then sends requests to the MySQL Database to retrieve the user's transaction data, financial goals, and historical analysis information. After receiving the data, the system processes the financial information, calculates a readiness score, and generates visualizations such as charts and analytics. The dashboard with these visual insights is then displayed to the user. If the user chooses to view a different time range or chart type, the system updates the charts and refreshes the dashboard accordingly. If the data retrieval fails or is incomplete, the system shows a warning message and displays partial results instead of the full dashboard.

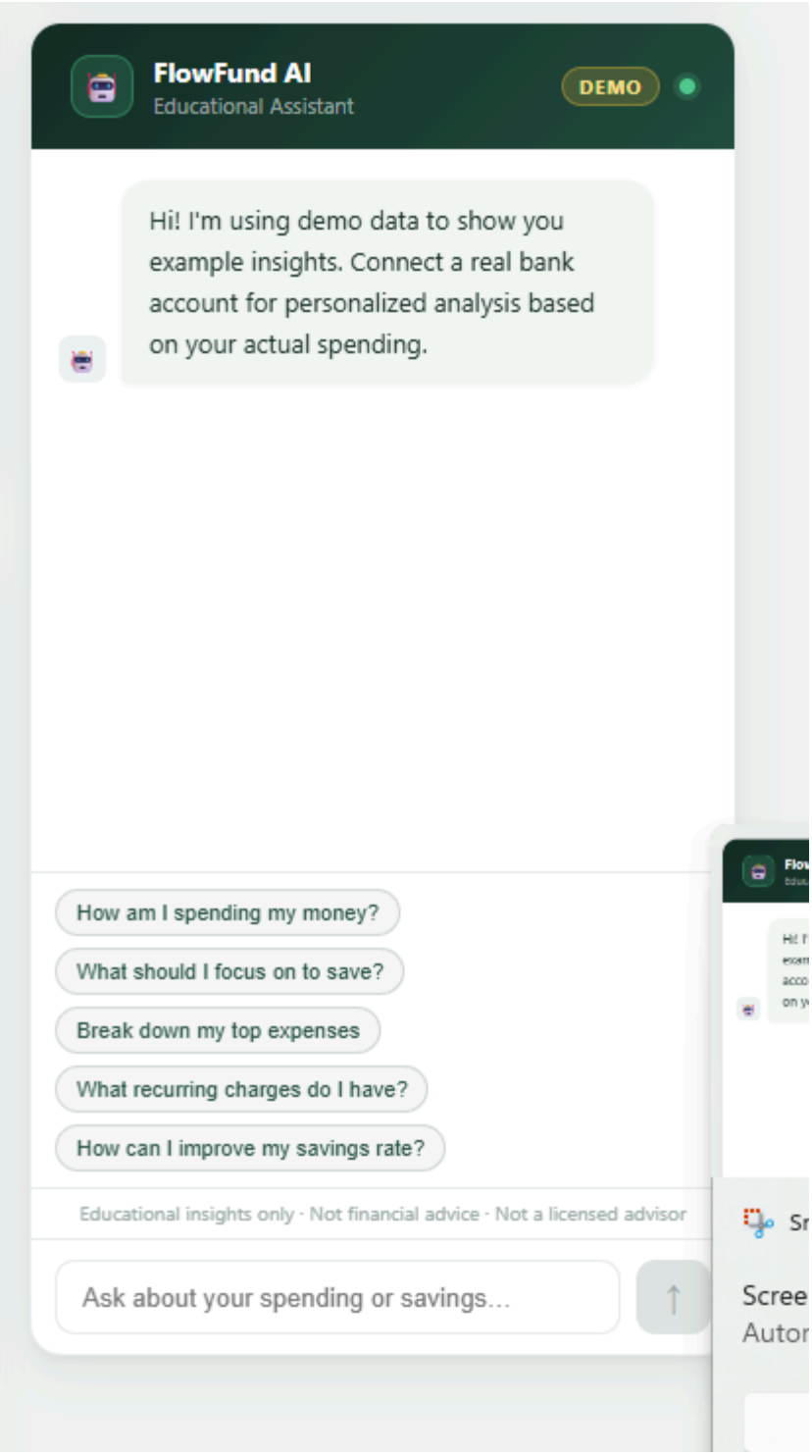

User
FlowFund System
MySQL Database
openDashboard()
requestTransactionData(userId)
transactionData
requestGoals(userId)
goalData
requestHistoricalAnalysis(userId)
analysisData
processFinancialData()
calculateReadinessScore()
generateVisualizations()
displayDashboard(charts, analytics)
alt
[User selects different time range or chart type]
changeVisualization(range/type)
updateCharts()
refreshDashboard()
alt
[Data retrieval fails or incomplete]
error / partialData
showWarningMessage()
displayPartialResults()

# Section 9: Implmentation

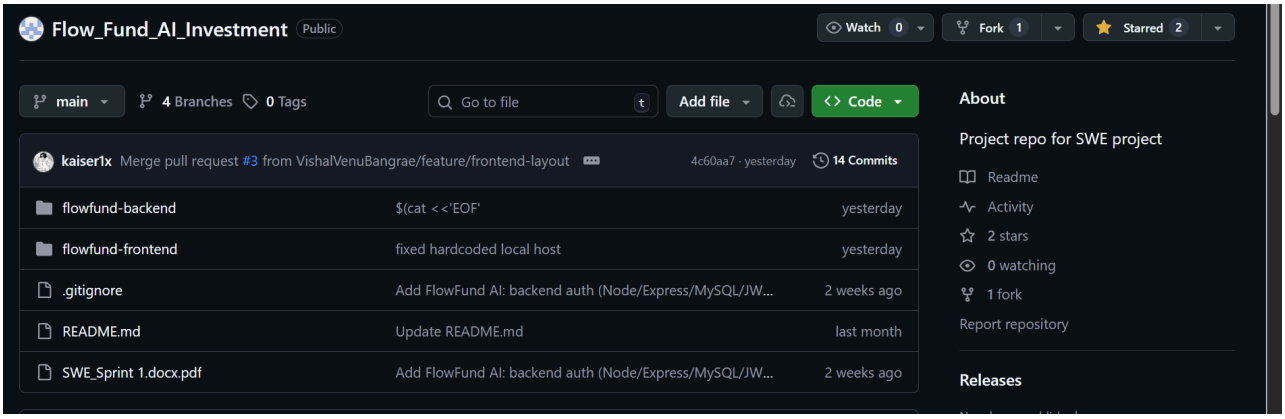



FLOWFUND AI
FINANCIAL PLATFORM
Dashboard
Market Analysis
Demo Mode
JD
John Doe
Log out
Dashboard
Monitor your financial health and investment readiness
TOTAL BALANCE
$320.00
2 accounts
MONTHLY SPEND
$508.55
Last 30 days
SAVINGS RATE
79%
Of monthly income
TOP CATEGORY
Groceries
Highest spend
Bank Accounts
2 connected
+ Connect Bank
Bank of America — Plaid Checking
CHECKING · ****0000
$110.00
Bank of America — Plaid Saving
SAVINGS · ****1111
$210.00
JD
John Doe
user
EMAIL
admin@example.com
CONNECTED
2 bank accounts
FlowFund AI
Educational Assistant
DEMO
Hi! I'm using demo data to show you example insights. Connect a real bank account for personalized analysis based on your actual spending.
Recent Transactions
20 transactions
DEMO
TODAY
Starbucks
Food & Drink
-$6.45
Uber Eats
Food & Drink
-$24.50
YESTERDAY
Amazon
Walmart
Groceries
-$67.30
Ask about your spending or savings...
Spending Breakdown
Last 30 days by category
DEMO
Groceries
$120.50
24%
Shopping
$90.35
18%
Education
$89.00
18%
Transportation
$76.50
15%
Food & Drink
$57.75
11%




Github Link: https://github.com/VishalVenuBangrae/Flow_Fund_AI_Investment.git
Github:


Flow_Fund_AI_Investment
Public
Watch 0
Fork 1
Starred 2
main
4 Branches
0 Tags
Go to file
t
Add file
Code
kaiser1x Merge pull request #3 from VishalVenuBangrae/feature/frontend-layout
4c60aa7 · yesterday
14 Commits
flowfund-backend
$(cat <<'EOF'
yesterday
flowfund-frontend
fixed hardcoded local host
yesterday
.gitignore
Add FlowFund AI: backend auth (Node/Express/MySQL/JW...
2 weeks ago
README.md
Update README.md
last month
SWE_Sprint 1.docx.pdf
Add FlowFund AI: backend auth (Node/Express/MySQL/JW...
2 weeks ago
About
Project repo for SWE project
Readme
Activity
2 stars
0 watching
1 fork
Report repository
Releases

| Assignee Name | Email address | Task | Duration (hours) | Dependency | Due Date | Evaluation |
|---|---|---|---|---|---|---|
| Ayan Lakhani | alakhani18@student.gsu.edu | Diagrams and Diagram explanations | 1 hr | Worked with Vrushti | 4/6/26 | 8/10 |
| Vrushti Patel | vpatel131@student.gsu.edu | Test cases and Architecture Modeling | 2 hr | Worked with Kirtan | 4/6/26 | 9/10 |
| Kirtan Patel (Coordinator) | kpatel315@student.gsu.edu | Improved Requirements/ Architectural Modeling Diagram | 2 hr | Worked with Vrushti | 4/6/26 | 10/10 |
| Quang Tran | qtran30@student.gsu.edu | Integrated frontend and backend | 3 hr | Vishal | 4/6/26 | 8/10 |
| Vishal Venu Bangrae | vbangrae1@student.gsu.edu | Updating backend and frontend, implementing table design. | 2 hr | Quang | 4/6/26 | 9/10 |

## Section 10: Testing

### Section 10.1: Test Cases

**Test Case 1: Login**

**Test Case 1.1: Successful Login**

**Test Inputs:**

- Email: valid_user@gmail.com
- Password: correct_password

**Expected Results:**

- Login successful
- Dashboard displayed

**Dependencies:**

- User exists

**Initialization:**

- Login page

**Test Steps:**

1. Enter a valid email
2. Enter the correct password
3. Click login

**Tear Down:**

- Logout

**Test Case 1.2: Incorrect Password**

**Test Inputs:**

- Email: valid_user@gmail.com
- Password: wrong_password

**Expected Results:**

- Error message displayed
- Login denied

**Dependencies:**

- User exists

**Initialization:**

- Navigate to login

**Test Steps:**

1. Enter a valid email
2. Enter the wrong password
3. Click login

**Tear Down:**

- None

**Test Case 1.3: Invalid Email Format**

**Test Inputs:**

- Email: invalidemail
- Password: any_password

**Expected Results:**

- Error message: "Invalid email format."
- Login not attempted

**Dependencies:**

- Login page available

**Initialization:**

- Navigate to the login page

**Test Steps:**

1. Enter an invalid email
2. Enter password
3. Click login

**Tear Down:**

- None

**Test Case 1.4: Email Not Registered**

**Test Inputs:**

- Email: new_user@gmail.com
- Password: any_password

**Expected Results:**

- Error: "User does not exist."
- Login denied

**Dependencies:**

- Email not in database

**Initialization:**

- Open the login page

**Test Steps:**

1. Enter a valid email not in the system
2. Enter password
3. Click login

**Tear Down:**

- None

**Test Case 2: Link Back Account**

**Test Case 2.1: Successful Bank Linking**

**Test Inputs:**

- Bank: Plaid Sandbox
- Username: valid_user
- Password: valid_password

**Expected Results:**

- Bank account linked successfully
- Access token stored securely
- User redirected to dashboard

**Dependencies:**

- Plaid API available
- User is authenticated

**Initialization:**

- Navigate to the "Link Bank Account" page

**Test Steps:**

1. Click "Link Bank"
2. Enter valid credentials
3. Select account
4. Confirm linking

**Tear Down:**

- Optionally unlink account

**Test Case 2.2: Invalid Bank Credentials**

**Test Inputs:**

- Username: invalid_user
- Password: wrong_password

**Expected Results:**

- Error message displayed
- Bank account not linked
- No data stored

**Dependencies:**

- Plaid API available

**Initialization:**

- Navigate to the bank linking page

**Test Steps:**

1. Click "Link Bank"
2. Enter invalid credentials
3. Submit

**Tear Down:**

- None

**Test Case 2.3: User Cancels Bank Linking**

**Test Inputs:**

- User clicks cancel during the Plaid flow

**Expected Results:**

- Linking process stops
- No account linked
- User returned to the dashboard or linking page

**Dependencies:**

- Plaid flow initiated

**Initialization:**

- Start the bank linking process

**Test Steps:**

1. Click "Link Bank"
2. Begin authentication
3. Click "Cancel"

**Tear Down:**

- None

**Test Case 3: Import Transactions**

**Test Case 3.1: Successful Transaction Linked**

**Test Inputs:**

- Bank: Plaid Sandbox
- Account: Checking (linked)

**Expected Results:**

- Transactions are retrieved successfully
- Transaction data, like amount, date, and category are stored in the database
- Transactions are displayed on the dashboard

**Dependencies:**

- Bank account linked
- Plaid API available

**Initialization:**

- User logged in
- Navigate to the dashboard

**Test Steps:**

1. Click "Refresh Transactions"
2. System fetches data from Plaid
3. Transactions are displayed

**Tear Down:**

- None

**Test Case 3.2: Account Not Linked**

**Test Inputs:**

- No bank account linked

**Expected Results:**

- Message displayed: “No bank account linked.”
- No transactions fetched
- The system does not crash

**Dependencies:**

- User logged in

**Initialization:**

- Navigate to the dashboard

**Test Steps:**

1. Click "Refresh Transactions"
2. System checks account status
3. Message is displayed

**Tear Down:**

- None

**Test Case 4: View Dashboard**

**Test Case 4.1: Dashboard with data**

**Test Inputs:**

- User with transaction data.

**Expected Results:**

- Dashboard loads successfully
- Charts and summaries are displayed (spending, categories)
- Data shown matches stored transactions

**Dependencies:**

- Transaction data exists
- User is logged in

**Initialization:**

- Navigate to the dashboard

**Test Steps:**

1. Open dashboard
2. System loads financial data
3. Charts and data are displayed

**Tear Down:**

- None

**Test Case 4.2: Dashboard without data**

**Test Inputs:**

- New user with no transactions

**Expected Results:**

- Message displayed: “No data available.”
- Dashboard loads without errors
- No charts are shown

**Dependencies:**

- User account exists

**Initialization:**

- Log in and navigate to the dashboard

**Test Steps:**

1. Open dashboard
2. System checks for data
3. Message is displayed

**Tear Down:**

- None

**Test Case 5: Investment Readiness Score**

**Test Case 5.1: High Score**

**Test Inputs:**

- Income: $5000
- Expenses: $1500
- Savings: $8000

**Expected Results:**

- High score generated (e.g., above 80)
- Message displayed: “Ready to invest.”
- Score shown on dashboard

**Dependencies:**

- Financial data available
- User logged in

**Initialization:**

- Navigate to the readiness module

**Test Steps:**

1. Open the readiness section
2. The system calculates the score
3. Score and message are displayed

**Tear Down:**

- None

**Test Case 5.2: Moderate Score**

**Test Inputs:**

- Income: $3000
- Expenses: $2500
- Savings: $1000

**Expected Results:**

- Moderate score generated
- Message displayed: “Improve savings before investing.”

**Dependencies:**

- Financial data available

**Initialization:**

- Navigate to the readiness module

**Test Steps:**

1. Open the readiness section
2. The system calculates the score
3. Results displayed

**Tear Down:**

- None

**Test Case 5.3: No Financial Data**

**Test Inputs:**

- No transactions or linked account

**Expected Results:**

- Message displayed: “Insufficient data.”
- No score generated
- System remains stable

**Dependencies:**

- User logged in

**Initialization:**

- Navigate to the readiness module

**Test Steps:**

1. Open the readiness section
2. System checks data
3. Message is displayed

**Tear Down:**

- None

**Section 10.2**

**Testing (JUnit)**

**Next sprint**

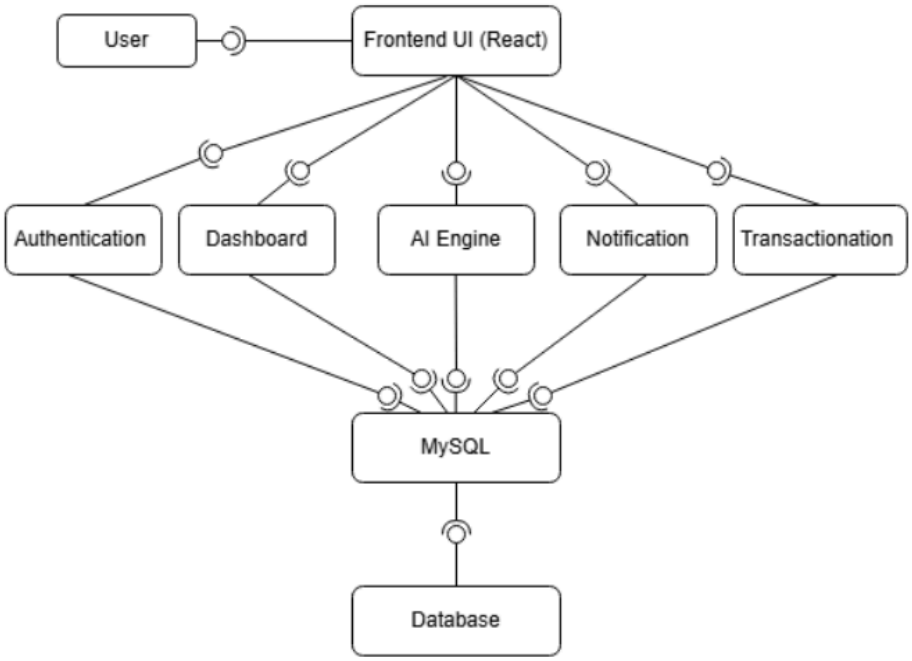
**Section 11: Architectural Modeling**

The system follows a Layered Architecture, which is a very suitable architectural model for financial website applications. This architecture separates the system into distinct layers, including the presentation layer, which is the frontend, the application layer, which is the backend, and the data layer, which is a database, along with integration of external services such as the Plaid API. This separation improves scalability, maintainability, and security of the system.

1. **Logical View**

   Logical View describes the functional structure of the system and the relationships between its core components. It focuses on how the system is logically organized to fulfill user requirements.

   In the FlowFund, the main entities are User, Bank Account, Transaction, Dashboard, and Investment Readiness Score. A User can link one or more Bank Accounts, each containing multiple Transactions categorized by your spending. These transactions and your earnings are used to generate an Investment Readiness Score. The Dashboard, which retrieves transaction data and displays financial insights, including the investment readiness score.

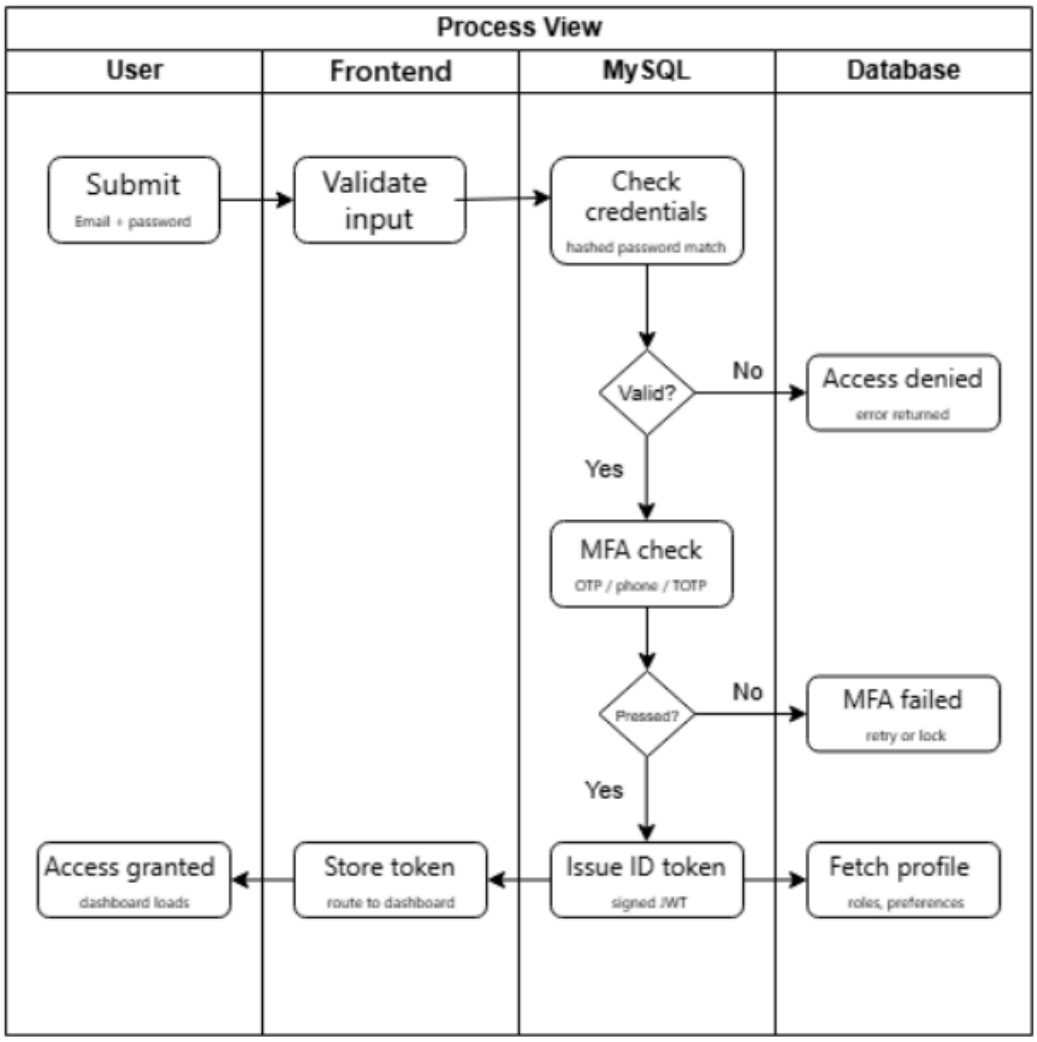



https://drive.google.com/file/d/1HH9BzjWYdcxjqBLHYQA6viWY6sifRAvC/view?usp=sharing

2. **Process View**

Process View describes the dynamic behavior of the system during execution. It shows how different components interact with each other to perform specific tasks.

In FLowFund, when a user links a bank account, the frontend sends a request to the backend. The backend communicates with the Plaid API to retrieve transaction data. Once the data is received, it is processed and stored in the database. The backend then sends the processed information back to the frontend, where it is displayed on the dashboard. This will emphasize the flow of data and the sequence of interactions between system components.

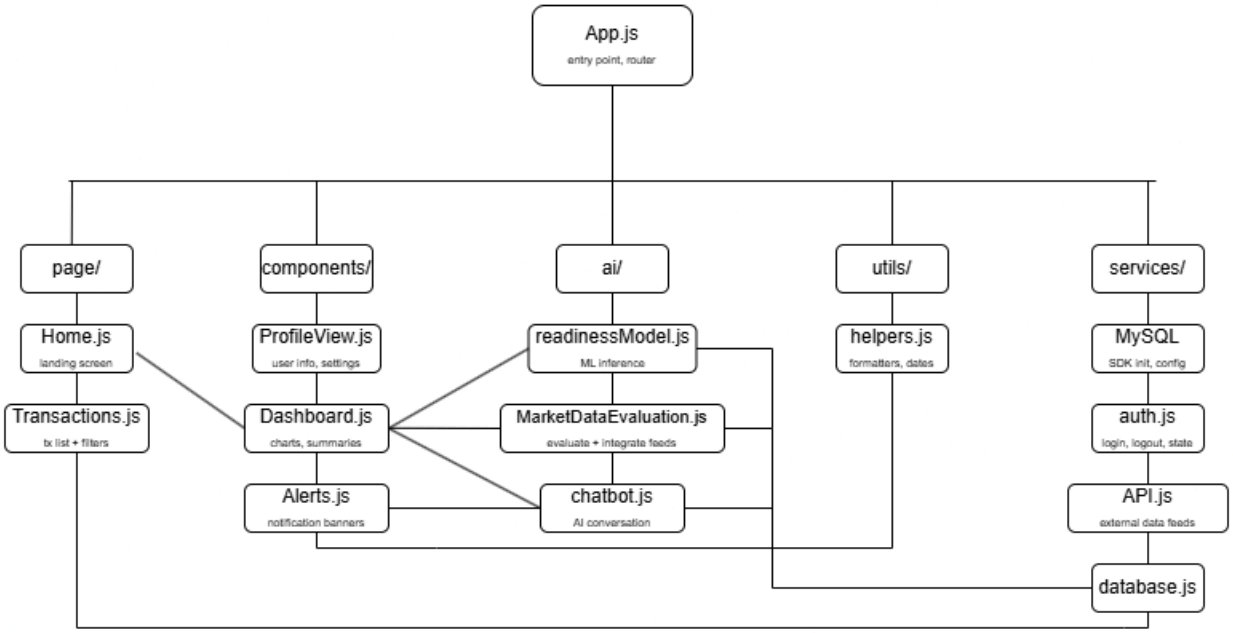



https://drive.google.com/file/d/15zfUzpy9UKSCh0n5lZJRk92QDsZXMyqS/view?usp=sharing

### 3. Development View

Development View illustrates how the system is structured from a programmer's perspective. It focuses on the organization of code into modules and components.

FlowFund is divided into a frontend and a backend. The frontend is implemented using a web framework such as React, which is responsible for user interaction and visualization. The backend contains several modules, including Authentication in User Management, Transaction Management, and Investment Readiness Scoring. These modules handle business logic and communicate with the database. In addition to that, the backend integrates with external services such as the Plaid API to fetch bank transaction data. This modular structure makes it easy to develop, test, and future enhancements.

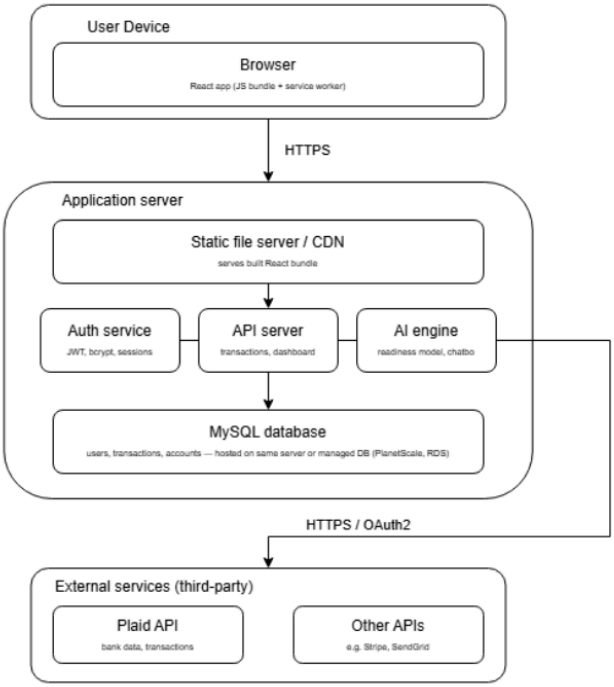



https://drive.google.com/file/d/1bTnX3ZvsqTFTGEKZTLCsc4wpWeSKALTy/view?usp=sharing

4. **Physical View**

Physical View represents the deployment of the system across hardware and network components. It shows where each part of the system resides and how they are connected.

In FlowFund, the user interacts with the application through a web browser. The frontend is hosted on a web server, which communicates with the backend application server. The backend server processes requests and interacts with a MySQL database for data storage. In addition to that, the backend communicates with external servers such as the Plaid API for financial data integration. This view ensures that the system is properly distributed and can handle real-world deployment requirements

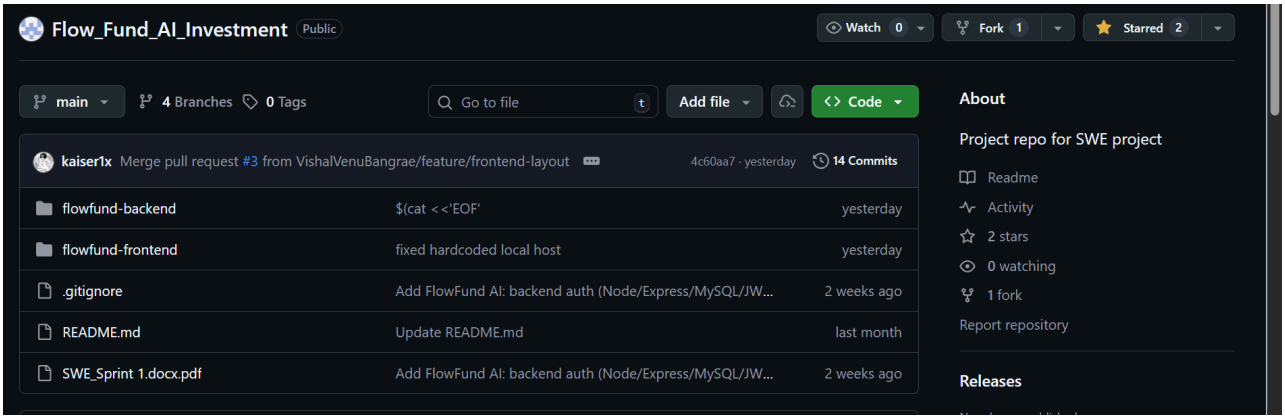


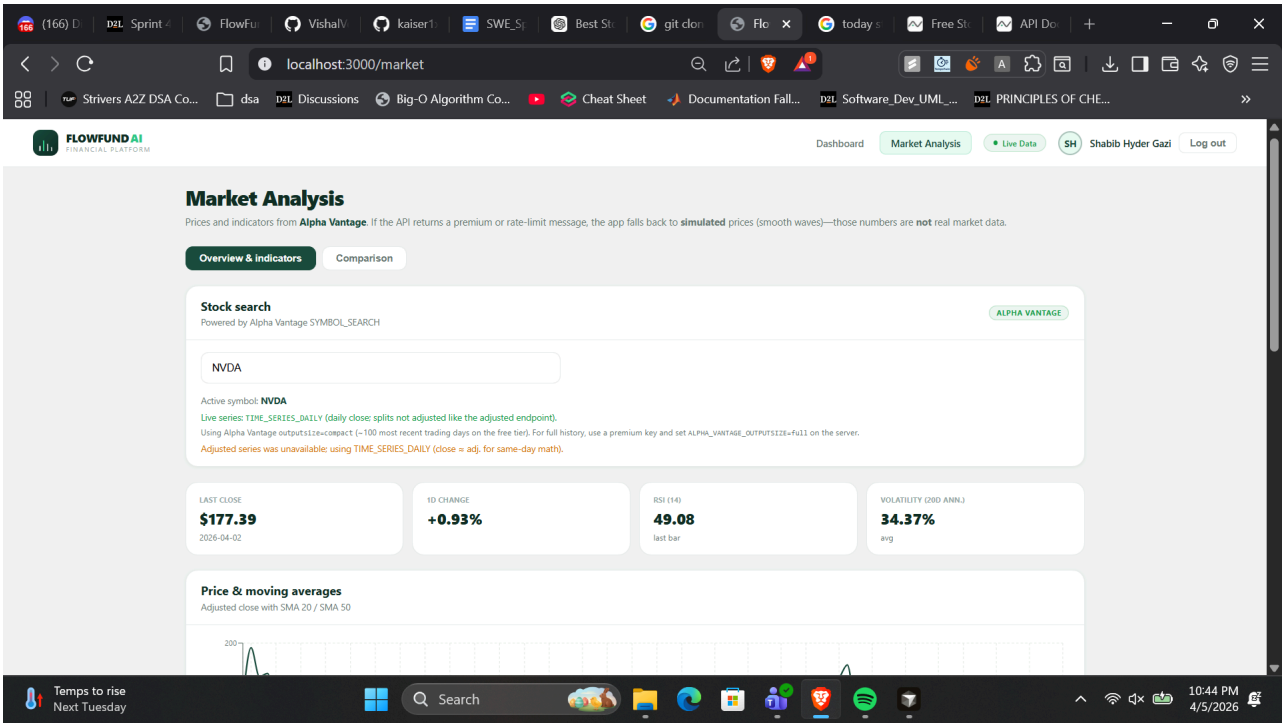

https://drive.google.com/file/d/1zL3kC8SCYPS-Jje-VDDJgC5J04Mj6XIm/view?usp=sharing

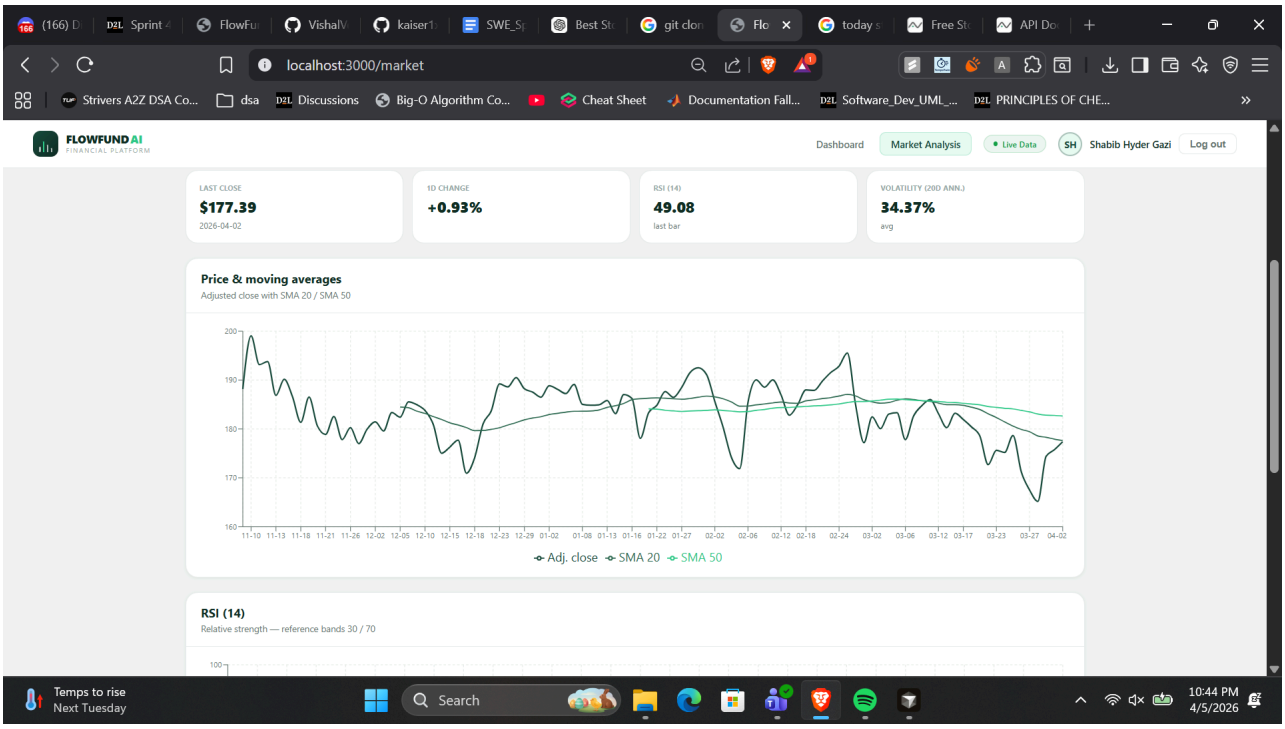
# GitHub link and Screenshot:

Github Link: https://github.com/VishalVenuBangrae/Flow_Fund_AI_Investment.git

Github Screenshot:




localhost:3000/market
FLOWFUND AI
FINANCIAL PLATFORM
Dashboard
Market Analysis
Live Data
Shabib Hyder Gazi
Log out
Market Analysis
Prices and indicators from Alpha Vantage. If the API returns a premium or rate-limit message, the app falls back to simulated prices (smooth waves)—those numbers are not real market data.
Overview & indicators
Comparison
Stock search
Powered by Alpha Vantage SYMBOL_SEARCH
ALPHA VANTAGE
NVDA
Active symbol: NVDA
Live series: TIME_SERIES_DAILY (daily close; splits not adjusted like the adjusted endpoint).
Using Alpha Vantage outputsize=compact (~100 most recent trading days on the free tier). For full history, use a premium key and set ALPHA_VANTAGE_OUTPUTSIZE=full on the server.
Adjusted series was unavailable; using TIME_SERIES_DAILY (close = adj. for same-day math).
LAST CLOSE
$177.39
2026-04-02
1D CHANGE
+0.93%
RSI (14)
49.08
last bar
VOLATILITY (20D ANN.)
34.37%
avg
Price & moving averages
Adjusted close with SMA 20 / SMA 50
Temps to rise
Next Tuesday
Search
10:44 PM
4/5/2026


localhost:3000/market
FLOWFUND AI
FINANCIAL PLATFORM
Dashboard
Market Analysis
Live Data
Shabib Hyder Gazi
Log out
LAST CLOSE
$177.39
2026-04-02
1D CHANGE
+0.93%
RSI (14)
49.08
last bar
VOLATILITY (20D ANN.)
34.37%
avg
Price & moving averages
Adjusted close with SMA 20 / SMA 50
200
190
180
170
160
Adj. close
SMA 20
SMA 50
RSI (14)
Relative strength — reference bands 30 / 70
100
Temps to rise
Next Tuesday
Search
10:44 PM
4/5/2026

## References


- U.S. Securities and Exchange Commission. (n.d.). *Investor Bulletin: What You Should Know About Robo-Advisers*. Retrieved [access date], from https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-45
- OWASP Foundation. (n.d.). *OWASP Cheat Sheet Series*. Retrieved 1/25/26, from https://cheatsheetseries.owasp.org/index.html